

1 π_{RL} : 面向基于 flow 的 Vision-Language-Action 模型的在线 RL 微调 (fine-tuning)

项目主页: <https://github.com/RLinf/RLinf> <https://huggingface.co/RLinf>

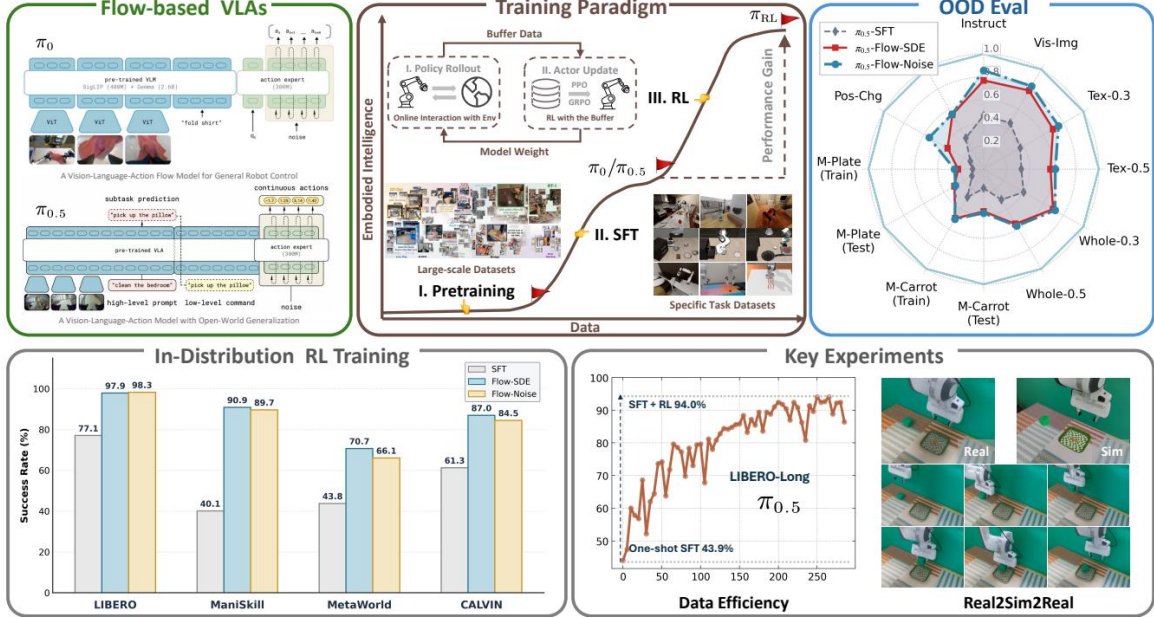


图 1: π_{RL} : 一个面向基于 flow 的 VLA 的在线 RL 框架。通过引入 Flow-Noise 和 Flow-SDE 两种方案, π_{RL} 在大规模 ID 基准测试和 OOD 设定下, 提升了经 SFT 对齐模型的性能与泛化能力。经过 RL 优化后, 仅使用少样本 SFT 得到的 policy, 便可达到与使用全量数据训练的 baseline 相当的性能。此外, 作者构建了一个以 3D Gaussian Splatting 为渲染引擎的模拟器, 以缩小视觉域差距, 从而支持无缝的 zero-shot sim-to-real 迁移。

2 Abstract

Vision-Language-Action (VLA) 模型使机器人能够基于多模态输入理解并执行复杂任务。尽管近期已有工作尝试利用强化学习 (RL) 来自动化数据采集这一繁重过程, 从而扩展监督微调 (SFT) 的规模, 但由于 flow matching 会导致动作 log-likelihood 难以计算, RL 仍难以直接应用于大规模、基于 flow 的 VLA (如 π_0 、 $\pi_{0.5}$)。为解决这一问题, 我们提出 π_{RL} , 其核心包含两项技术方案: (1) Flow-Noise: 将去噪过程建模为一个离散时间 MDP, 并引入可学习的噪声网络, 以实现精确的 log-likelihood 计算; (2) Flow-SDE: 将去噪过程与智能体—环境交互进行联合建模, 构造一个双层 MDP, 并通过 ODE 到 SDE 的转换实现高效的 RL exploration。我们在多个 benchmark 上评估了 π_{RL} , 实验表明, RL 在分布内与分布外两种设置下都能带来显著的性能提升。

3 引言

Vision-Language-Action (VLA) 模型 (Din et al., 2025) 已成为通用机器人领域的一类代表性方案, 能够有效弥合高层多模态推理与底层物理控制之间的鸿沟 (Firoozi et al., 2025)。在传感器输入和语言指令的条件下, VLA (Team et al., 2024; Kim et al., 2024; Black et al., 2024; Intelligence et al., 2025) 可以将抽象指令转化为可执行的机器人动作, 从而实现更直观、更灵活的人机交互。

如图 1 所示, VLA 的训练通常遵循标准的预训练与监督微调 (SFT) 范式。具体而言, 在预训练 Vision-Language Model (VLM) (Touvron et al., 2023; Beyer et al., 2024) 的基础上, VLA 先在大规模、异构的人类 demonstration 数据集上进行微调 (O’Neill et al., 2024; Khazatsky et al., 2024), 随后再针对目标任务执行 SFT, 使其能力与特定 embodiment 及环境相适配。然而, 对 SFT 的依赖带来了一个关键挑战: 构建大规模、高质量的专家轨迹既耗费人力, 也成本高昂 (Din et al., 2025)。此外, 通过 SFT 获得的模型往往容易对专家 demonstration 过拟合 (Fei et al., 2025), 其性能上限也从根本上受制于专家 demonstration 的质量。

近期工作 (Zang et al., 2025; Li et al., 2025a; Tan et al., 2025; Liu et al., 2025a) 开始探索将强化学习 (RL) 引入 VLA 的训练流程, 形成如图 1 所示的“预训练 + SFT + RL”范式。借助与环境的交互, 这一范式使 VLA 有机会突破专家 demonstration 的性能边界, 并学习到泛化性更强的 policy。

然而, 现有这些 RL 进展主要局限于自回归式 VLA, 例如 OpenVLA (Kim et al., 2024) 和 OpenVLA-OFT (Kim et al., 2025)。这类方法采用离散动作 decoder, 以自回归或并行方式生成输出。相比之下, 流式 (flow-based) VLA 以 π 系列模型为代表, 它们通过 flow matching (Lipman et al., 2022) 中的迭代精炼来生成动作, 具有能够高频生成 action chunk, 并胜任高灵巧操作任务的优势 (Black et al., 2024)。因此, 已有的 VLA-RL 算法无法直接适配 flow-based VLA, 而其中的根本难点在于: 如何为已执行动作刻画其对数似然 (log-likelihood) (Hutchinson, 1989; Chen et al., 2018)。

为此, 本文提出 π_{RL} , 一个面向 flow-based VLA 的在线 RL 微调框架。针对 flow matching 中难以处理的对数似然估计问题, 我们提出了两种解决方案。其一, Flow-Noise 在去噪过程中引入一个可学习的噪声网络, 并将该阶段建模为离散时间的马尔可夫决策过程 (MDP), 从而实现精确的对数似然估计。其二, Flow-SDE 在保持边缘分布等价的前提下, 将基于常微分方程 (ODE) 的去噪过程转化为随机微分方程 (SDE), 以支持探索, 并进一步构建一个两层 MDP, 将去噪过程与 policy-环境交互耦合起来。在完成 MDP 形式化以及精确对数似然计算后, π_{RL} 可通过 proximal policy optimization (PPO) (Schulman et al., 2017) 继续优化。

我们在多个基准上进行了大量实验, 以评估 π_{RL} 在 π_0 (Black et al., 2024) 和 $\pi_{0.5}$ (Intelligence et al., 2025) 模型上的有效性。实验结果表明, 在所有基准上, 所提出框架都能相较于 SFT baseline 持续带来显著的性能提升。进一步地, 分布外评估表明, 该模型获得的是真正的 policy 增强, 而非仅在目标环境上的狭义过拟合。

总的来说, 本文贡献如下:

- **面向 flow-based VLA 的 RL。**我们提出了 π_{RL} , 这是一个用于 flow-based VLA 的在线 RL 微调框架, 其中包含 Flow-Noise 与 Flow-SDE 两种建模形式。
- **更优的性能表现。**我们在多个基准上验证了 π_{RL} 的显著性能提升及更强的泛化能力。
- **全面的消融研究。**我们进行了系统的 ablation 实验, 并据此给出了可为后续 flow-based VLA 的 RL 研究提供参考的实证分析。

4 Introduction (续)

- **开源代码与模型。**我们公开发布全部代码, 以确保结果可复现, 并希望本研究能够推动该领域的进一步发展。

5 2. Related Work

5.1 Vision-Language-Action Models

近年来, VLA 模型通过整合多模态输入, 在机器人领域取得了显著进展, 使感知、推理与控制能够在统一框架下协同完成。基于这一发展趋势, 研究社区提出了一系列代表性架构, 包括 Octo (Team et al., 2024)、RT (Brohan et al., 2022)、OpenVLA、OpenVLA-OFT、 π_0 、 $\pi_{0.5}$ 以及 GR00T (Bjorck et al., 2025)。

5.2 2.2. 面向 VLA 模型的在线 RL 微调

近年来, 越来越多的研究开始关注利用在线 RL 提升 VLA 的性能与泛化能力。例如, SimpleVLA-RL (Li et al., 2025a) 基于 OpenVLA-OFT 和 GRPO, 表明在数据稀缺条件下, RL 能够提升 VLA 模型的长时程规划能力。RL4VLA (Liu et al., 2025a) 在分阶段稀疏奖励设置下, 对 PPO、GRPO 以及 direct preference optimization (DPO) [?] 进行了实证评估。RLinf-VLA (Yu et al., 2025; Zang et al., 2025) 则提供了一个统一且高效的框架, 用于支持 VLA 模型可扩展的 RL 训练。这些工作共同表明, 采用 RL 对 VLA 模型进行微调是有效的。

5.3 2.3. Flow Models 的 RL 微调

将 RL 与 flow model 结合, 是突破模仿学习局限性的一条很有前景的路径。为此, Flow-GRPO (Liu et al., 2025b) 将确定性的 ODE 转换为等价的 SDE, 从而支持带随机性的探索; 在此基础上, 后续工作如 Mix-GRPO (Li et al., 2025b) 和 TempFlow-GRPO (He et al., 2025) 进一步采用混合 ODE-SDE rollout 来加速训练。ReinFlow (Zhang et al., 2025) 则在 flow 路径中注入可学习噪声, 并将其转化为具有可处理 likelihood 的离散时间 Markov 过程, 以实现稳定的 policy gradient 更新。Flow policy optimization (FPO) (McAllister et al., 2025) 将 policy optimization 重新表述为: 最大化 conditional flow matching loss 的 advantage 加权比值。

6 3. Preliminary

6.1 3.1 问题表述

我们将该任务建模为一个 MDP, 其定义为六元组 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P_0, P_{\text{ENV}}, R_{\text{ENV}}, \gamma)$ 。其中, 状态 $s_t \in \mathcal{S}$ 定义为机器人的观测 $\mathbf{o}_t$, P_0 表示初始状态分布。给定当前状态后, flow policy 预测动作 $a_t \sim \pi(\cdot|s_t) \in \mathcal{A}$, 进而产生状态转移 $s_{t+1} \sim P_{\text{ENV}}(\cdot|s_t, a_t)$, 并获得奖励 $R_{\text{ENV}}(s_t, a_t)$ 。我们的目标是学习一个 policy π_θ , 使其在长度为 $T+1$ 的时域上最大化 γ -折扣回报的期望:

$$\mathcal{I}(\pi_\theta) = \mathbb{E}_{\pi_\theta, P_0} \left[\sum_{t=0}^T \gamma^t R_{\text{ENV}}(s_t, a_t) \right]. \quad (1)$$

基于 policy gradient 的替代目标 (Williams, 1992), 回报期望关于参数的梯度可由采样轨迹近似为:

$$\nabla_\theta \mathcal{J}(\pi_\theta) = \mathbb{E}_{\pi_\theta, P_0} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) A(s_t, a_t) \right]. \quad (2)$$

其中，优势函数 $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$ 用于衡量动作价值 $Q(s_t, a_t)$ 相对于状态价值 $V(s_t)$ 的相对优越性，从而为 policy 更新提供一个低方差的学习信号。

6.2 3.2. 基于 Flow 的 Vision-Language-Action Model

基于 Flow 的 VLA 模型 π_θ 旨在将观测 $\mathbf{o}_t$ 映射为长度为 H 的未来动作序列 $\mathbf{A}_t = [a_{t,0}, \dots, a_{t,H-1}]$ 。其中， $\mathbf{o}_t$ 由 RGB 图像、语言 token 以及机器人的 proprioception 构成，该建模可写为 $p(\mathbf{A}_t|\mathbf{o}_t)$ 。在该模型中，VLM 负责从视觉与语言输入中提取特征，而 flow matching expert 则负责生成动作。具体而言，模型学习一个条件向量场 $\mathbf{v}_\theta$ ，将标准高斯噪声分布逐步变换为目标动作 $\mathbf{A}_t$ 。这一过程通过最小化条件 Flow Matching (CFM) 损失来实现，即使预测向量场 $\mathbf{v}_\theta$ 与真实向量场 $\mathbf{u}$ 对齐：

$$[\text{formula}] \tag{3}$$

这里，条件概率路径 $q(\mathbf{A}_t^\tau|\mathbf{A}_t)$ 在 flow matching 中根据动作 $\mathbf{A}_t$ 、随机噪声 $\epsilon \sim \mathcal{N}(0, I)$ 以及连续时间 $\tau \in [0, 1]$ ，生成带噪动作¹ $\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau)\epsilon$ 。对于该特定路径，对应的真实向量场定义为 $\mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t) = \mathbf{A}_t - \epsilon$ 。

在推理阶段，动作序列的生成首先从噪声向量 $\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$ 开始采样，随后基于前向 Euler 方法，在固定步数上对所学习的向量场 $\mathbf{v}_\theta$ 进行积分迭代，从而逐步细化动作序列：

$$\mathbf{A}_t^{\tau+\delta} = \mathbf{A}_t^\tau + \mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) \cdot \delta$$

7 4. 方法

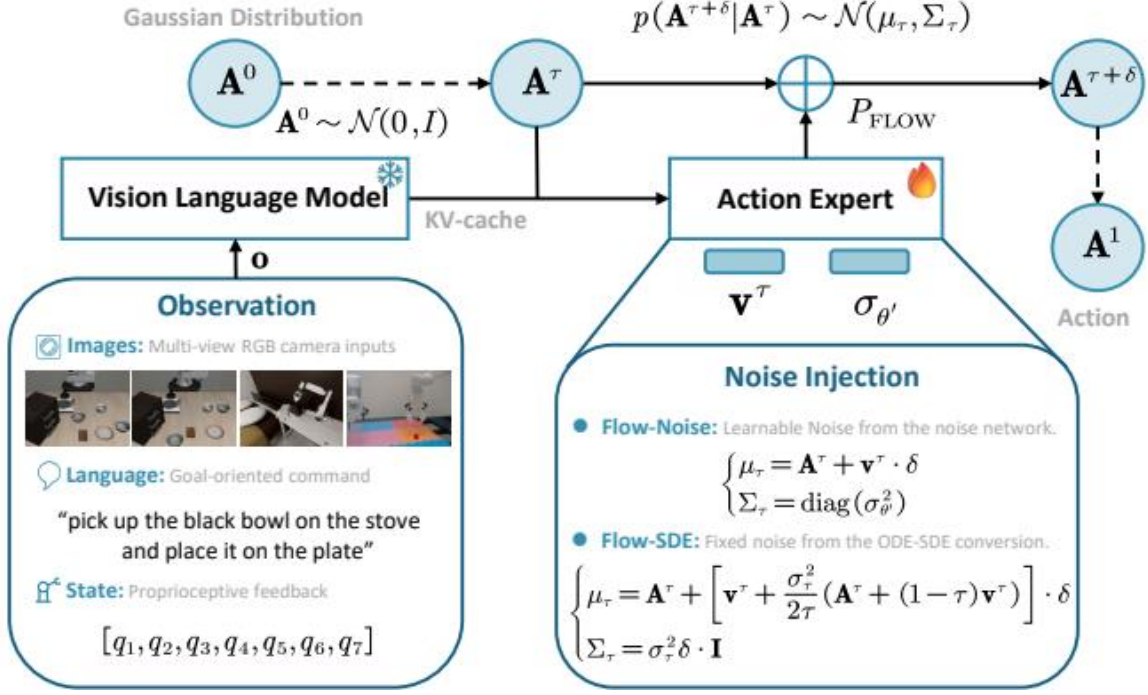
现有的 VLA-RL 方法通常基于不同的基础模型：离散动作场景多采用 OpenVLA，连续动作场景则多采用 OpenVLA-OFT。为计算动作对数似然 $\log \pi_\theta(a_t|\mathbf{s}_t)$ ，离散模型 (Liu et al., 2025a) 通常对输出 logits 施加 softmax；连续模型 (Li et al., 2025a) 则将动作建模为高斯分布，并使用预测头来估计方差。

对于基于 flow 的 VLA，若直接计算其精确似然 (Hutchinson, 1989)，在 denoising 步数较少时往往不够准确。此外，由于其基于 ODE 的采样过程是确定性的，模型缺乏探索能力，因此很难直接嵌入 RL 框架。为此，我们提出了 Flow-Noise 和 Flow-SDE 两种技术方案，使基于 flow 的 VLA 能够适用于 RL。

7.1 4.1. Flow-Noise

受 Reinflow (Zhang et al., 2025) 启发，我们在 flow matching 的去噪过程中引入了一个可学习的噪声网络，并在第 3.1 节所述的标准单层 MDP 框架下求解该问题。我们将去噪阶段建模为一个离散 MDP，因此可以直接计算去噪后序列的 log-likelihood，并进一步通过 RL 实现等价的 policy 优化。

¹原文为 noisy action。

图 2: π_{RL} 中噪声注入的示意图

7.2 4.1.1. 随机性注入

在 Flow-Noise 中，我们使用一个神经网络对噪声调度进行参数化，使得训练过程中注入噪声的幅度能够被动态学习，从而提升灵活性，如图 2 所示。这里我们关注单个环境时间步 t 内的生成过程。为简化记号，省略时间下标 t ，例如将 $\mathbf{A}_t^\tau$ 写作 $\mathbf{A}^\tau$ ，并将预测速度 $\mathbf{v}_\theta(\mathbf{A}^\tau, \mathbf{o})$ 记为 $\mathbf{v}^\tau$ 。

在去噪过程中，步间状态转移被建模为高斯分布 $p(\mathbf{A}^{\tau+\delta} | \mathbf{A}^\tau) \sim \mathcal{N}(\mu_\tau, \Sigma_\tau)$ ，其中，均值由原始 ODE 的前向 Euler 更新决定，方差则由可学习的噪声网络 θ' 控制：

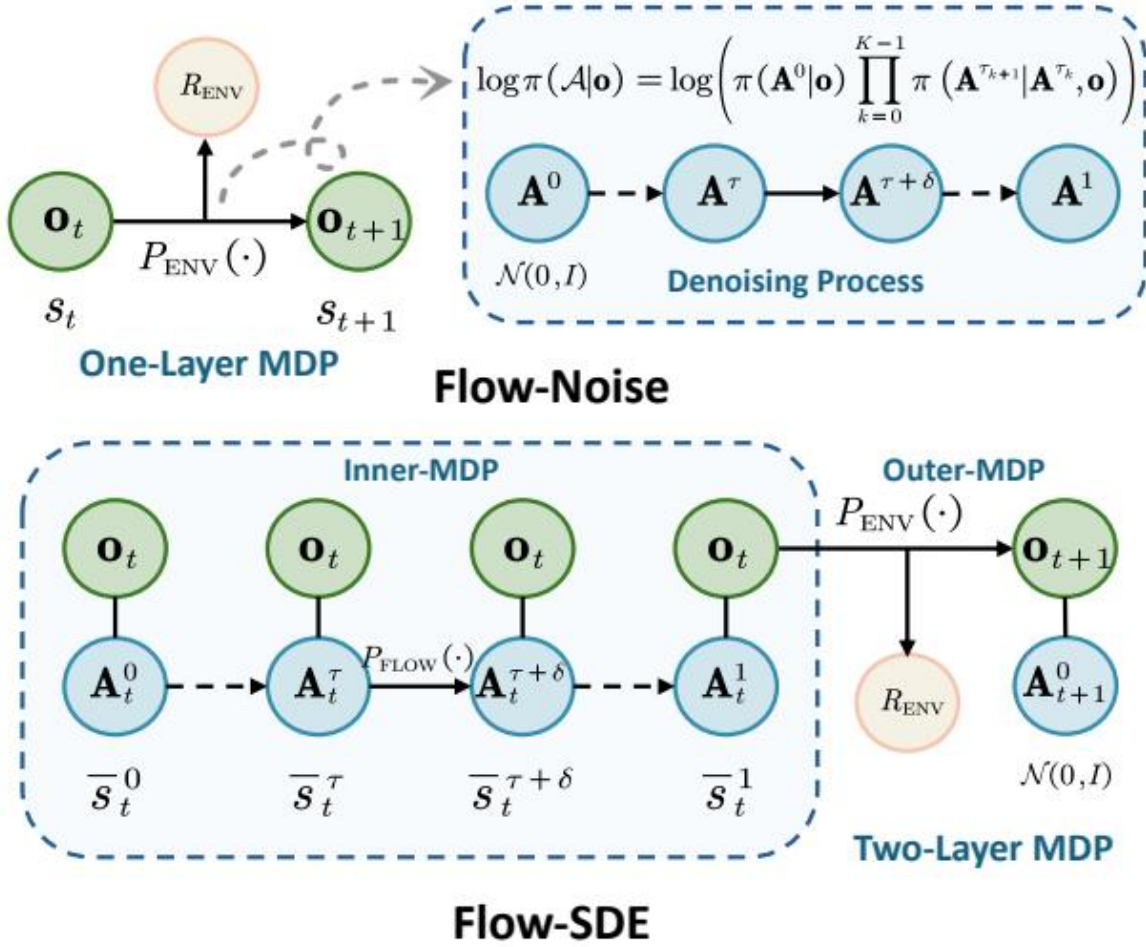
$$[\text{formula}]. \quad (4)$$

其中， $\sigma_{\theta'}(\cdot)$ 表示由噪声注入网络学习得到的标准差，其条件输入为动作 $\mathbf{A}^\tau$ 和观测 $\mathbf{o}$ 。该噪声网络与速度网络联合训练，但在微调（fine-tuning）完成后会被丢弃，因此推理阶段保留的是一个确定性 policy。

7.3 4.1.2. 对数似然估计

将 policy gradient 方法应用于基于 flow 的 VLA 时，核心难点在于：最终实际执行动作的对数似然不可 tractable。对此，Flow-Noise 的做法是，将整个 denoising 过程的联合对数似然梯度替换到式 (2) 的 policy 优化目标中；这一处理具有 Reinfo (Zhang et al., 2025) 的理论支撑。

在动作生成的推理过程中，我们将其离散化为 K 个均匀步，从而定义时间点序列 $\{\tau_0, \tau_1, \dots, \tau_K\}$ 。令步长间隔为 $\delta = 1/K$ ，则第 k 个离散时刻可写为 $\tau_k = k \cdot \delta$ ，其中起点为 $\tau_0 = 0$ ，终点为 $\tau_K = 1$ 。给定观测 $\mathbf{o}$ ，整个 denoising 序列 $\mathcal{A} = (\mathbf{A}^0, \dots, \mathbf{A}^1)$ 的精确且可 tractable 的对数概率如图 3 所示，其形式为：

图 3: π_{RI} 中 MDP 形式化的示意图

$$\log \pi(\mathbf{A}|\mathbf{o}) = \log \left(\pi(\mathbf{A}^0|\mathbf{o}) \prod_{k=0}^{K-1} \pi(\mathbf{A}^{\tau_{k+1}}|\mathbf{A}^{\tau_k}, \mathbf{o}) \right) \quad (5)$$

在此基础上，我们可以将基于 flow 的 policy 优化纳入标准 MDP 框架中进行处理。

7.4 4.2. Flow-SDE

受 Flow-GRPO (Liu et al., 2025b) 启发，我们将去噪过程从 ODE 形式改写为 SDE 形式，以增强随机探索能力。进一步地，参考 DPPO (Ren et al., 2024)，我们构建了一个双层 MDP，用于耦合去噪过程与 policy 和环境之间的交互；同时结合混合 ODE-SDE 采样技术，以加速训练过程。

7.4.1 随机性注入

在 Flow-SDE 中，我们将确定性的 ODE 转换为与之等价的 SDE，同时保持所生成动作的边缘概率密度不变，如图 2 所示。

flow matching，尤其是 Rectified Flow [?], 其确定性 ODE 采样轨迹可由前向 Euler 方法描述为：

$$d\mathbf{A}^\tau = \mathbf{v}^\tau d\tau. \quad (6)$$

基于 probability flow ODE 与 SDE 之间的联系 [?], 我们可以将式 (6) 中的确定性 ODE 改写为一个等价的 SDE: 其中 drift term 用于修正原始速度场, diffusion term 则负责注入噪声:

$$d\mathbf{A}^\tau = \underbrace{\left(\mathbf{v}^\tau - \frac{1}{2}g^2(\tau)\nabla \log q_\tau(\mathbf{A}^\tau) \right)}_{\text{Drift Term}} d\tau + \underbrace{g(\tau)d\mathbf{w}}_{\text{Diffusion Term}}, \quad (7)$$

其中, $g(\tau)$ 是控制噪声调度的标量函数, $\nabla \log q_\tau(\mathbf{A}^\tau)$ 表示边缘分布 q_τ 的 score function, $d\mathbf{w}$ 表示 Wiener 过程。

正如 Flow-GRPO 所建立的, score function 与速度场之间存在关键联系: $\nabla \log q_\tau(\mathbf{A}^\tau) = -\frac{\mathbf{A}^\tau}{\tau} - \frac{1-\tau}{\tau}\mathbf{v}^\tau$ 。将该 score function 用速度场表示后代入式 (7), 并将噪声调度 $g(\tau)$ 设为 $\sigma_\tau = a\sqrt{\frac{\tau}{1-\tau}}$, 其中 a 用于控制噪声强度, 即可得到 flow-matching sampler 的最终 SDE 形式:

$$d\mathbf{A}^\tau = \left[\mathbf{v}^\tau + \frac{\sigma_\tau^2}{2\tau} (\mathbf{A}^\tau + (1-\tau)\mathbf{v}^\tau) \right] d\tau + \sigma_\tau d\mathbf{w}_\tau. \quad (8)$$

对该 SDE 进行离散化可以看出, 转移概率 $p(\mathbf{A}^{\tau+\delta}|\mathbf{A}^\tau) \sim \mathcal{N}(\mu_\tau, \Sigma_\tau)$ 服从各向同性高斯分布, 其均值与方差为:

$$[\text{formula}] \quad (9)$$

7.5 4.2.2. MDP 形式化

在 Flow-SDE 中, 我们将 flow matching 的去噪过程与环境交互耦合起来。具体而言, 我们把去噪过程中定义的内部 MDP, 嵌入到第 3.1 节中带有环境 $\mathcal{M}_{\text{ENV}}$ 的高层外循环 MDP 内, 从而构造出一个如图 3 所示的双层 MDP。其各个组成部分同时相对于环境时间 t 与去噪时间 τ 定义。

- 状态 $\bar{s}_t^\tau = (\mathbf{o}_t, \mathbf{A}_t^\tau)$ 由观测 $\mathbf{o}_t$ 与动作状态 $\mathbf{A}_t^\tau$ 组成。
- 动作 $\bar{a}_t^\tau$ 被定义为: 在内循环中, 下一个采样得到的去噪动作; 在外循环中, 则是实际执行的动作:

$$\bar{a}_t^\tau = [\text{formula}], \quad (10)$$

其中, $\mathbf{A}_t^{\tau+\delta} = \mu_\tau + \sigma_\tau \sqrt{\delta} \cdot \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$ 表示随机采样得到的噪声。

- 转移 $\bar{P}(\bar{s}_t^{\tau'}|\bar{s}_t^\tau, \bar{a}_t^\tau)$ 描述状态如何演化, 其形式为:

$$[\text{formula}]. \quad (11)$$

当 $\tau < 1$ 时, 发生的是不同去噪动作状态之间的内循环转移 $P_{\text{FLOW}}(\cdot)$ 。此时观测 $\mathbf{o}_t$ 保持不变, 而下一个动作状态由 $\bar{a}_t^\tau = \mathbf{A}_t^{\tau+\delta}$ 给定。

当 $\tau = 1$ 时, 最终动作 $\bar{a}_t^\tau = \mathbf{A}_t^1$ 会与外循环环境发生交互, 并依据环境动力学 $P_{\text{ENV}}(\cdot)$ 产生新的观测 $\mathbf{o}_{t+1}$ 。与此同时, 动作状态会从标准正态分布重新初始化, 即 $\mathbf{A}_{t+1}^0 \sim \mathcal{N}(0, \mathbf{I})$ 。

- 奖励 $\bar{R}(\bar{s}_t^\tau, \bar{a}_t^\tau)$ 仅在去噪过程完成并与环境交互后才给出:

$$[\text{formula}]. \quad (12)$$

在这双层 MDP 框架下, 原本需要估计的动作对数似然 $\log \pi(a_t|s_t)$, 被转化为估计 $\log \pi(\bar{a}_t^\tau|\bar{s}_t^\tau)$ 。由于转移过程具有高斯性质, 这一量的计算是直接的。

7.6 4.2.3. 混合 ODE-SDE 采样

双层 MDP 表述会显著拉长决策时域，从而增加训练难度与计算开销。为缓解这一问题，我们采用混合 ODE-SDE rollout 策略 (Li et al., 2025b; He et al., 2025)。在每个时间步 t ，我们随机采样一个去噪时间 τ_t ，用于随机性的 SDE 探索；同时，将其余所有去噪步都视为确定性的 ODE 更新。

具体而言，policy 作用于状态 $\bar{s}_t^{\tau_t} = (\mathbf{o}_t, \mathbf{A}_t^{\tau_t})$ ；随后，由环境 wrapper 执行剩余的 ODE 步以及环境状态转移，最终在新采样的时间 τ_{t+1} 处得到下一状态 $\bar{s}_{t+1}^{\tau_{t+1}} = (\mathbf{o}_{t+1}, \mathbf{A}_{t+1}^{\tau_{t+1}})$ 。这种表述在保持与原始双层框架理论一致性的同时，有效缩短了 MDP 的时域。

7.7 4.3. Policy Optimization

7.7.1 4.3.1. ALGORITHM

在上述 flow policy MDP 形式化之后，我们的目标是为 policy π_θ 学习最优参数 θ^* ，使其期望折扣回报 $\mathcal{I}(\pi_\theta)$ 最大。为此，我们采用机器人与强化学习中广泛使用的 policy gradient 算法 PPO 来优化该 policy。

π -series 模型 (Black et al., 2024; Intelligence et al., 2025) 采用基于 chunk 的动作生成方式。具体而言，policy 在每次观测输入后，输出一个长度为 H 的未来动作序列 $\mathbf{A}_t = [a_{t,0}, \dots, a_{t,H-1}]$ 。在这种设定下，我们将整个序列视为一个宏步 (macro-step)，并将其对应奖励定义为 [formula]，即把各时间步奖励 $\mathbf{r}_{t,j}$ 求和。这一做法在 RLinf-VLA (Zang et al., 2025) 中被称为 chunk-level formulation。

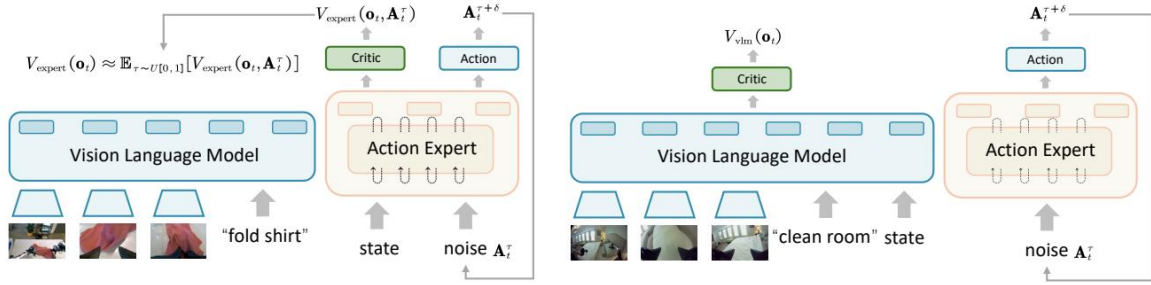


图 4: 两种 critic 放置配置的示意图。(a) 带 action expert 的 critic，以 π_0 为例；(b) 带 VLM 的 critic，以 $\pi_{0.5}$ 为例。

为了更有效地指导 policy 更新，PPO 使用广义优势估计 (Generalized Advantage Estimation, GAE) (Schulman et al., 2015) 来计算低方差的 advantage 估计，其形式为：

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma\lambda)^k \mathcal{T}_{t+k}, \quad (13)$$

其中，TD-error 定义为 $\mathcal{T}_t = R_t + \gamma V(s_{t+1}) - V(s_t)$ 。这里， $V(\cdot)$ 表示由 critic network 给出的状态价值函数， γ 是折扣因子， λ 用于权衡 advantage 估计中的偏差与方差。

PPO 会将 policy 更新限制在较小的 trust region 内，以避免过大的不稳定更新，其目标函数为：

$$[\text{formula}]$$

其中，clip 函数由超参数 ϵ 控制，将比率 $\rho_t(\theta)$ 限制在区间 $[1 - \epsilon, 1 + \epsilon]$ 内，从而保证训练稳定性。

这里，更新后 policy 与旧 policy 之间的概率比 $\rho_t(\theta)$ 可以写为以下两种形式之一：

$$\rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \text{ or } \rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(\bar{a}_t^\tau|\bar{s}_t^\tau)}{\pi_{\theta_{\text{old}}}(\bar{a}_t^\tau|\bar{s}_t^\tau)}. \quad (14)$$

7.8 4.3.2. Critic 设计

遵循 VLA-PPO 的相关工作(Zang et al., 2025; Liu et al., 2025a), 我们采用共享的 actor-critic 架构, 以实现更节省内存的 value prediction, 如图 4 所示。不过, 这两个基于 flow 的 VLA 对 proprioceptive state 的处理方式不同: 在 π_0 中, state 被输入到 action expert 模型; 而在 $\pi_{0.5}$ 中, state 则在 VLM 内部与 prompt embedding 融合。

基于这一差异, 对于 $\pi_{0.5}$ 变体, 我们将 critic 网络直接接在 VLM 输出之后, 从而得到 value estimate $V_{\text{vlm}}(\mathbf{o}_t)$, 其条件输入为融合后的图像、语言与状态信息。相对地, 对于 π_0 变体, value prediction 的实现并不直接, 因为其输入结构是耦合的: action expert 同时依赖带噪动作 $\mathbf{A}_t^\tau$ 和 state。为此, 我们通过在整体 denoising trajectory 上对 value estimate 取平均, 来近似 $V_{\text{expert}}(\mathbf{o}_t)$, 形式化表达为:

$$V_{\text{expert}}(\mathbf{o}_t) \approx \mathbb{E}_{\tau \sim U[0,1]}[V_{\text{expert}}(\mathbf{o}_t, \mathbf{A}_t^\tau)]. \quad (15)$$

8 5. Experimental Results

8.1 5.1. Setup

基准测试 (Benchmarks) 我们在四个被广泛采用的机器人操纵 benchmark 上开展实验: LIBERO (Liu et al., 2023)、ManiSkill (Tao et al., 2024)、MetaWorld (McLean et al., 2025) 以及 CALVIN (Mees et al., 2022)。

基于 flow 的 VLA 我们的主要实验基于 π_0 和 $\pi_{0.5}$ 两个模型开展。除此之外, 我们还在附录 H 节中对 GR00T 进行了实验, 结果验证了本文算法同样可以应用于其他基于 flow 的 VLA。

8.2 5.2. 主要结果

本节首先在多个基准上评估 π_{RL} 的分布内 (ID) 性能, 随后进一步分析其分布外 (OOD) 泛化能力。

8.3 5.2.1. 分布内 RL 训练

如表 1 所示, 在所有评测 benchmark 上, π_{RL} 相比 SFT baseline 都带来了显著的性能提升。具体而言, π_0 模型的平均性能提升最高达到 +29.2%, 而 $\pi_{0.5}$ 变体的平均成功率 (success rate) 提升达到 +31.0%。

在 LIBERO 上, 我们先对 $\pi_{0.5}$ 模型进行 few-shot SFT, 再执行 RL 优化, 最终取得了 98.3% 的成功率, 超过了使用完整数据集训练的 SFT baseline (96.9%)。这种性能增益同样出现在其他更具挑战性的环境中, 包括包含 4,352 种 pick-and-place 任务组合的 ManiSkill、包含 50 种不同 manipulation primitive 的 MetaWorld, 以及面向长时序顺序任务的 CALVIN。更完整的实验细节见附录 Sec. C。

表 1: 四个 benchmark 上分布内 (ID) 性能的综合比较

Model		Benchmarks					
		LIBERO	ManiSkill	MetaWorld	CALVIN	Avg.	Δ Avg.
π_0	SFT	57.6	38.4	50.8	57.5	51.1	
	Flow-SDE	96.1	78.8	78.1	61.7	78.7	+27.6
	Flow-Noise	97.6	77.8	85.8	59.9	80.3	+29.2
$\pi_{0.5}$	SFT	77.1	40.1	43.8	61.3	55.6	
	Flow-SDE	97.9	90.9	70.7	87.0	86.6	+31.0
	Flow-Noise	98.3	89.7	66.1	84.5	84.7	+29.1

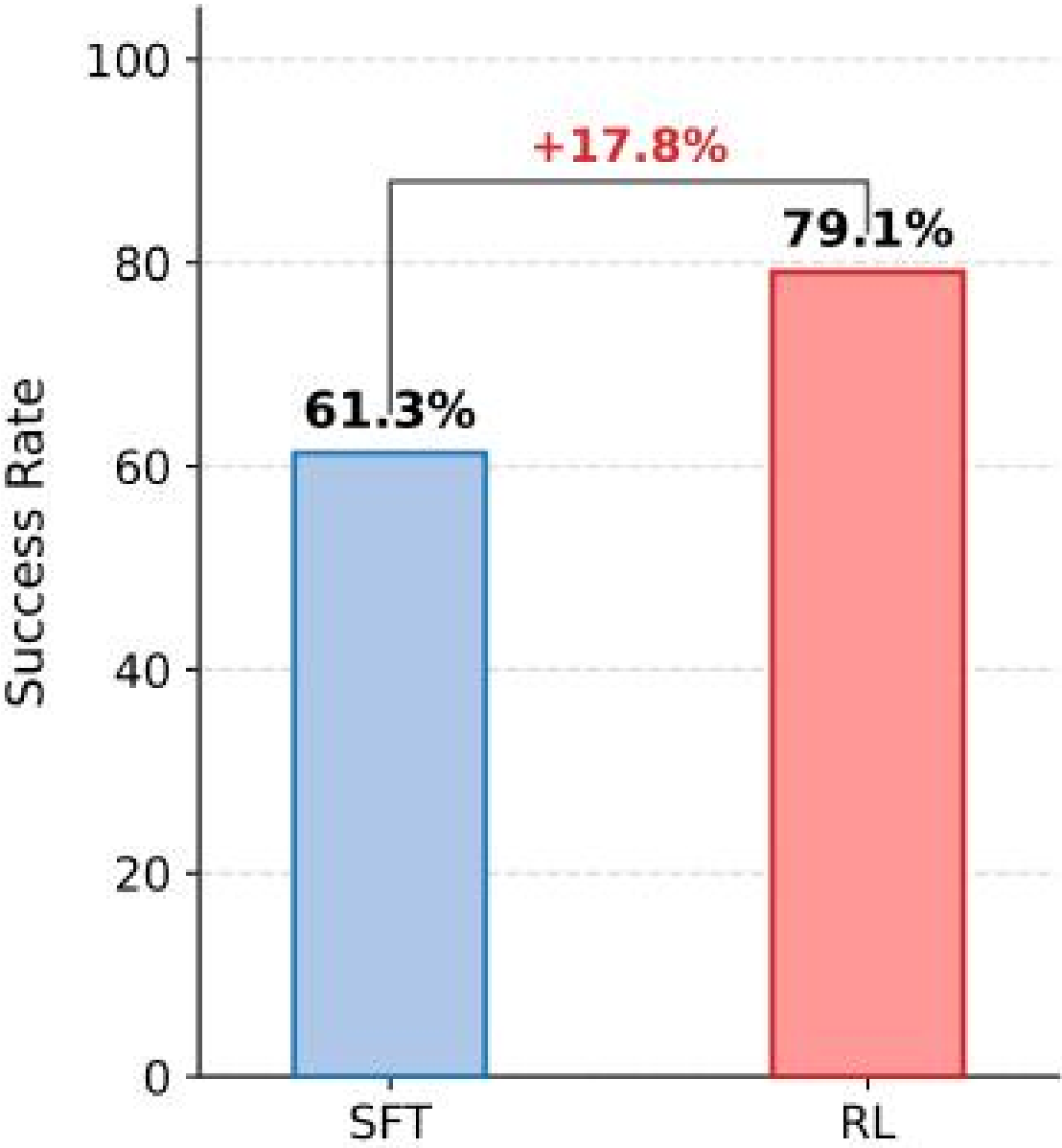


图 5: CALVIN 上的综合 OOD 评测结果。



图 6: ManiSkill OOD benchmark 上的综合评测结果。

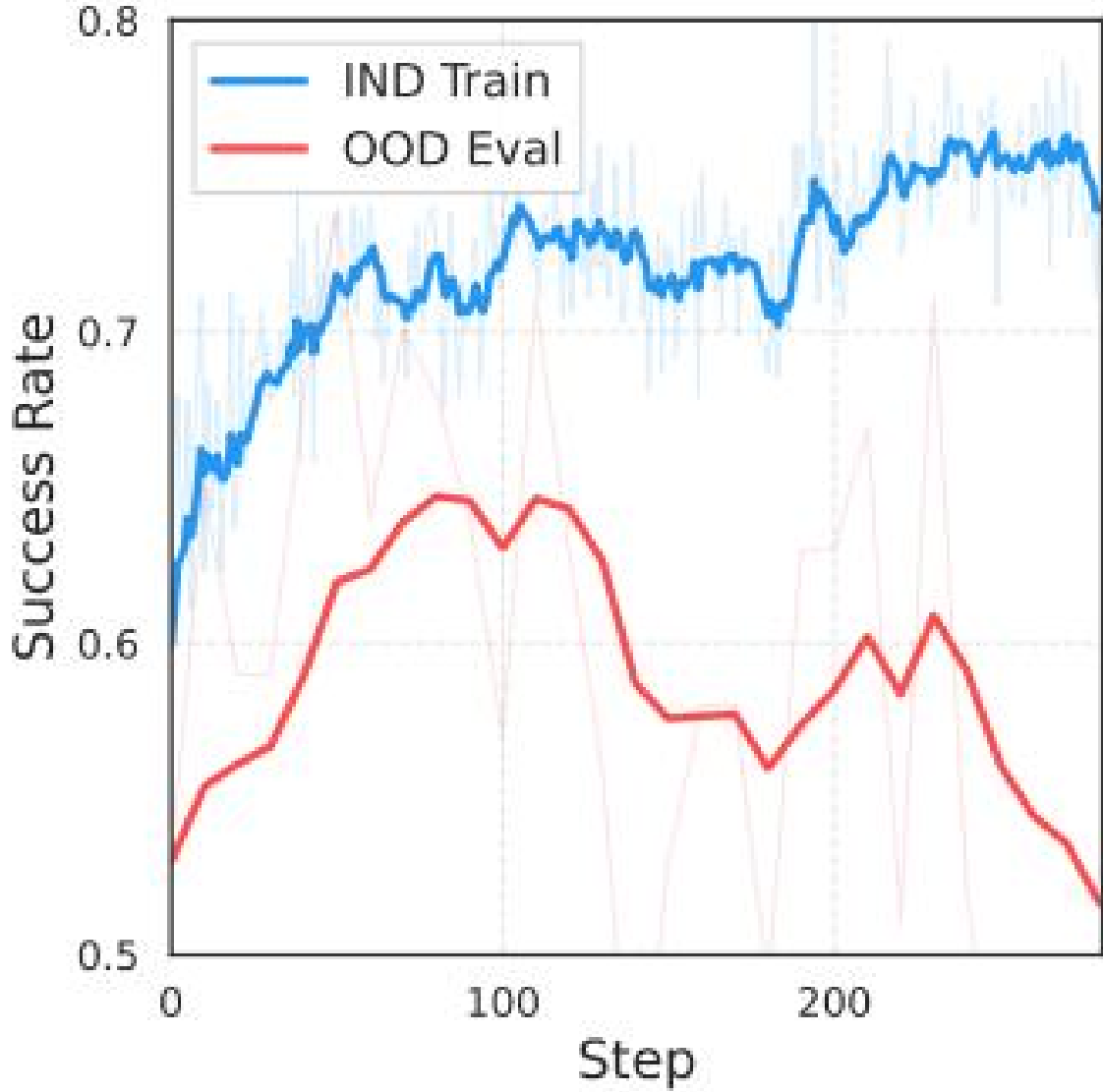


图 7: MetaWorld ML45 benchmark 上的综合评测结果。

图 5. CALVIN ABC-D、ManiSkill OOD 与 MetaWorld ML45 benchmark 上的综合 OOD 评测结果。

8.4 5.2.2. 分布外 (Out-of-Distribution, OOD) RL 评估

尽管前述实验表明，RL 能够带来性能提升，但一个关键问题仍然存在：RL 究竟是学到了更优的 policy，还是仅仅在给定奖励的驱动下对 ID 环境产生了过拟合？本节在 OOD 场景下评估经 RL 微调 (fine-tuning) 的 policy，其中环境分布或任务目标相较于 ID 训练设置发生偏离，以此检验其泛化能力。

如图 5 所示，在 ManiSkill 和 CALVIN 中，ID 设置下获得的性能增益能够有效迁移到 OOD 场景；此时的 domain shift 主要来源于环境变化。相比之下，MetaWorld 的 OOD 设置涉及不同的操纵任务，其性能虽然有所波动，但未表现出显著提升。该结果表明，RL 带来的收益主要体现在动作层面的精细化改进，而非更广泛的跨任务泛化能力增强。更多细节见附录 D 节。

8.5 5.3. Ablation Study

考虑到 Flow-SDE 在计算效率更高的同时，能够达到与 Flow-Noise 相当的性能，我们在消融实验中采用 Flow-SDE 方法。具体而言，我们考察 critic 设计、噪声注入策略以及 MDP 表述方式的影响；关于 RL 算法与超参数的更多实验结果见附录第 F 节。

8.6 5.3.1. Critic 设计

放置位置 (Placement) 我们在 LIBERO-Long 任务集上，以 π_0 模型比较了两种 critic 放置策略：一种接在 action expert 之后，对应 V_{expert} ；另一种接在 VLM 之后，对应 V_{vlm} 。如图 6 所示，尽管 V_{vlm} 不接收 proprioception 状态作为输入，但其整体表现仍略优，具体体现在更低的 value loss 和更高的 explained variance 上。

这一优势主要源于两者输入形式的关键差异： V_{vlm} 学习的是从 observation 到 value 的直接映射；而 V_{expert} 则需要同时处理耦合的状态输入与带噪声的动作输入，因此其优化更具挑战。

不过，为了与 value function 的概念保持一致，我们在 π_0 中仍保留了 V_{expert} 的架构，从而确保状态信息被纳入 value 估计过程。

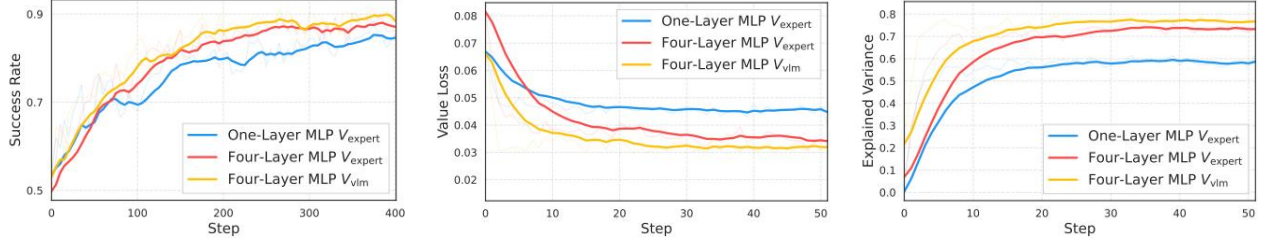


图 8: Flow-SDE π_0 在 LIBERO-Long 上关于 critic 设计的消融实验。(a) Eval; (b) Value Loss; (c) Explained Variance。结果表明，接在 VLM 之后的 critic，即 V_{vlm} ，表现更优。此外，在 V_{expert} 中，四层 MLP 展现出更强的回归能力。

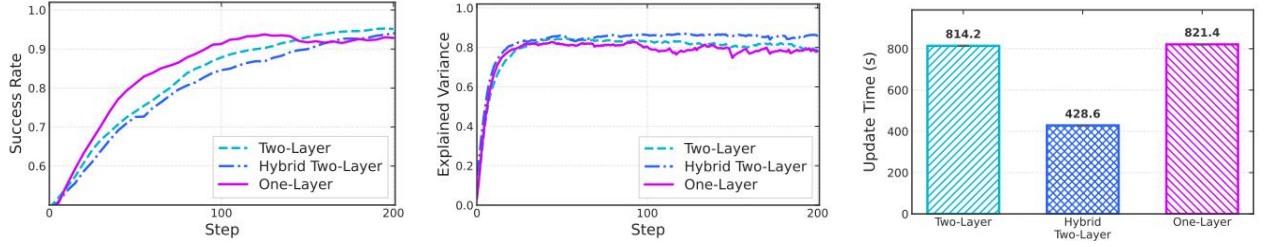


图 9: Flow-SDE π_0 在 LIBERO-Goal 上关于 MDP formulation 的消融实验。(a) Eval; (b) Explained Variance; (c) Update Time。

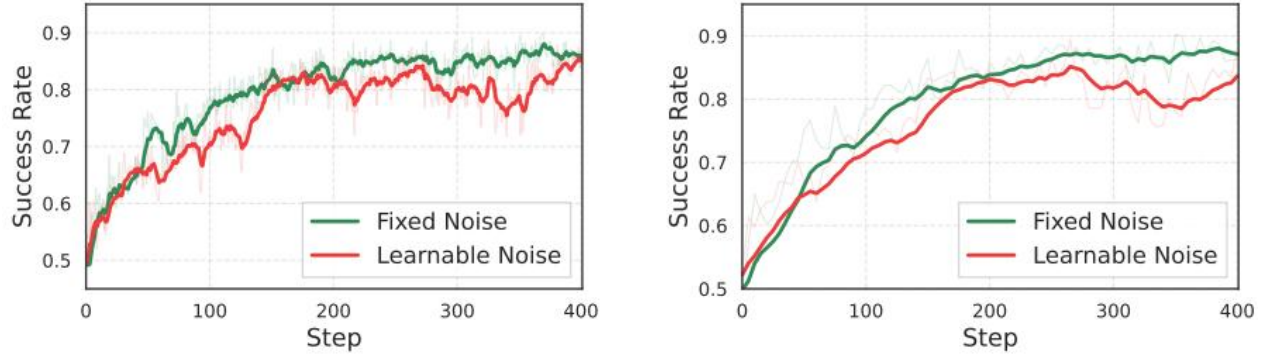


图 10: Flow-SDE π_0 在 LIBERO-Long 上关于注入策略 (injection strategy) 的消融实验。(a) Train; (b) Eval。

结构 (Structure) 我们进一步比较了四层 MLP 与单层 MLP。后者与 action expert 中的 action-projection 结构一致。图 6 的结果表明，四层 MLP 能得到更准确的 value 近似，从而带来更好的性能与更稳定的训练过程。

8.7 5.3.2. Flow Policy MDP

在采用相同固定噪声注入策略的前提下，我们在 LIBERO-Goal 上比较了 Flow-Noise 的单层 MDP 与 Flow-SDE 的双层 MDP，如图 7 所示。

可以观察到，单层形式的收敛速度最快，但三种形式最终达到的成功率保持一致。从计算效率来看，混合式双层范式由于有效 MDP 链更短，相比标准方法实现了 $2\times$ 的加速。需要指出的是，单层 MDP 相比标准双层模型并未带来显著的 wall-clock time 优势，其原因在于：为了进行 log likelihood 估计，仍然需要重新计算完整的去噪轨迹。

8.8 5.3.3. 随机性注入

我们基于 Flow-SDE MDP 形式，在 LIBERO-Long 基准上比较了固定噪声注入与可学习噪声注入两种策略。为保证比较可控，对于可学习噪声，我们将 entropy bonus 设为 0，使其与固定噪声方案保持一致。

具体而言，我们将固定噪声设为 0.5，并将可学习噪声的 log-variance 下界和上界分别设为 0.08 和 0.16。图 8 表明，在 step 0 时，两种噪声策略的训练表现相近，说明二者对应的噪声幅值具有可比性。进一步地，最终收敛结果也验证了这两种注入方式都具有效率。

表 2: Flow-SDE 在 LIBERO-Spatial 上的超参数消融实验。Train 表示随机 rollout 阶段的 policy 性能，Eval 表示确定性评估阶段的性能。

Models	Stage	Hyperparameters									
		Noise Level			Denoise Step				Action Chunk		
		0.2	0.5	0.8	1	2	4	8	5	10	20
SFT	Train	62.3 / 65.2	56.0 / 65.2	46.6 / 65.2	9.4	28.3 / 64.9	56.1 / 65.2	62.6	56.0 / 65.2	60.7	70.3 / 72.6
	Eval				63.8			63.2		70.5	
RL	Train	59.5	93.5	95.3	73.8	90.8	93.5	84.3	93.5	93.3	87.5
	Eval	73.1	94.5	98.1	88.5	97.0	94.5	86.7	94.5	95.5	89.2

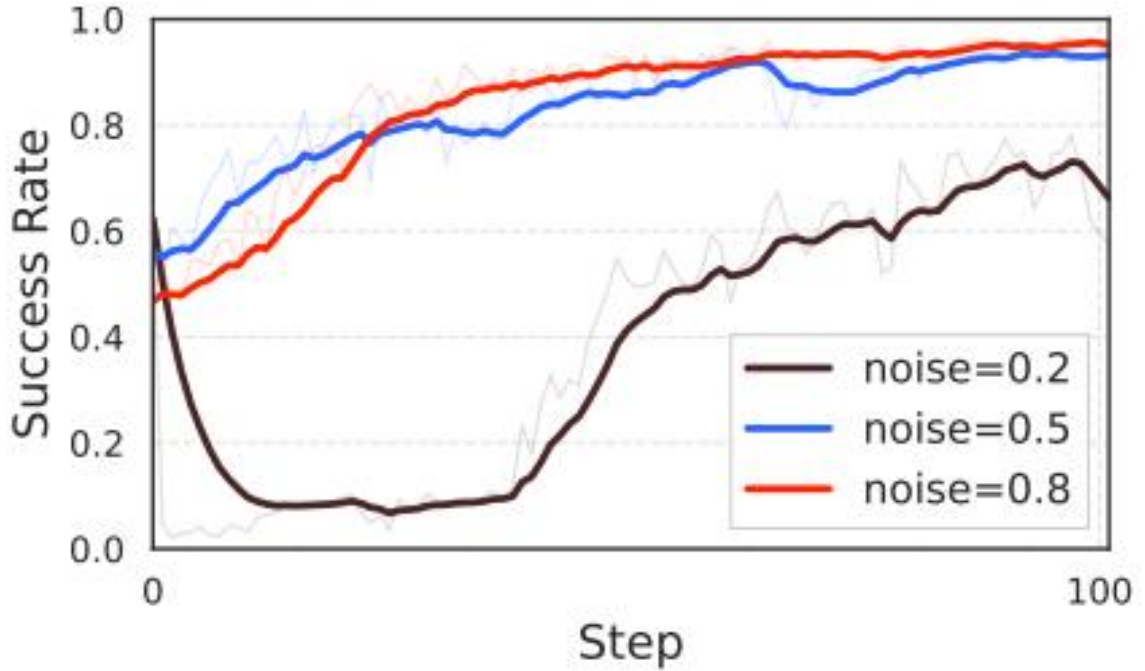


图 11: 噪声水平 a 的消融实验: Train 结果。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

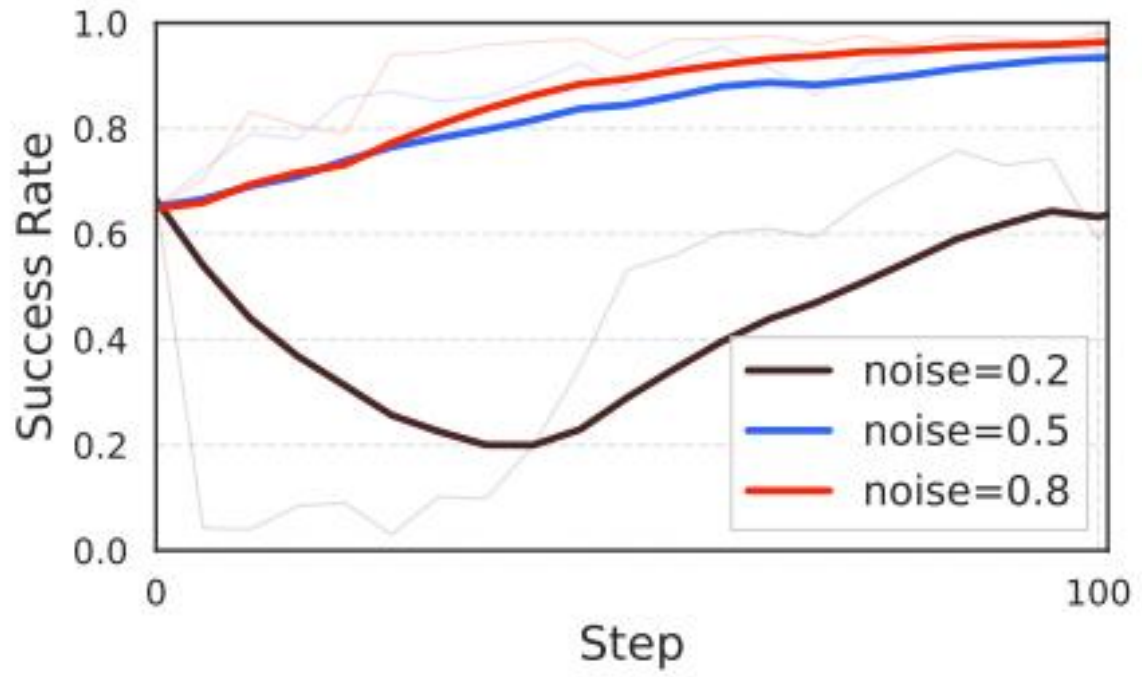


图 12: 噪声水平 a 的消融实验: Eval 结果。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

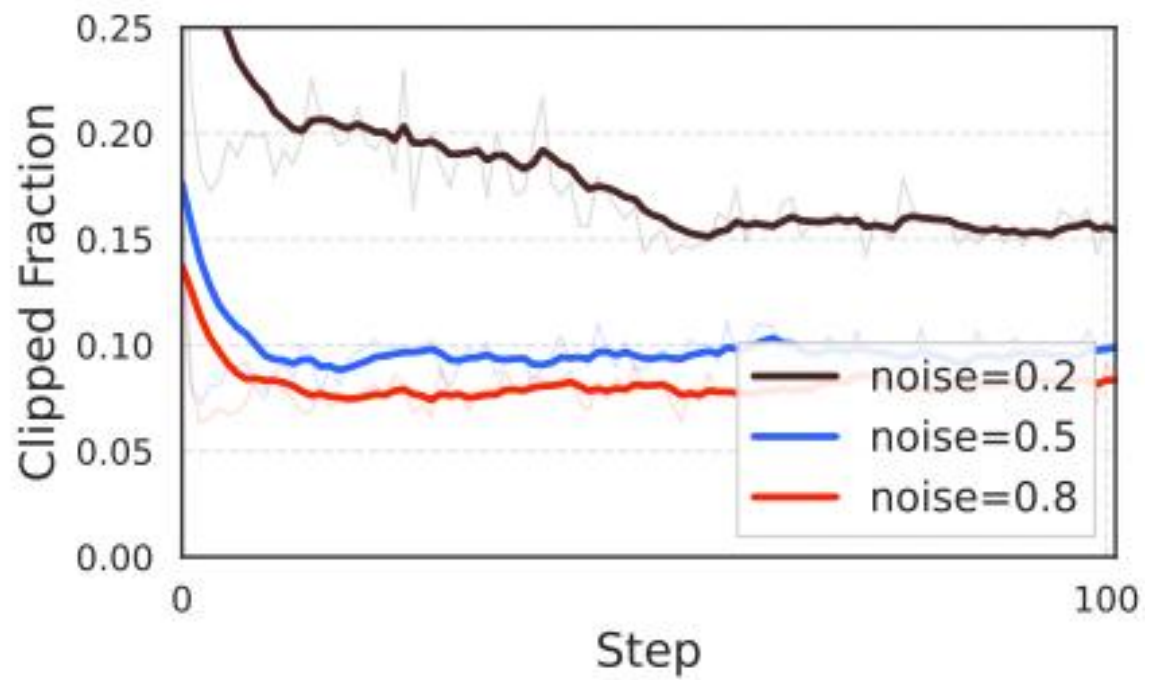


图 13: 噪声水平 a 的消融实验: Clipped Fraction。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

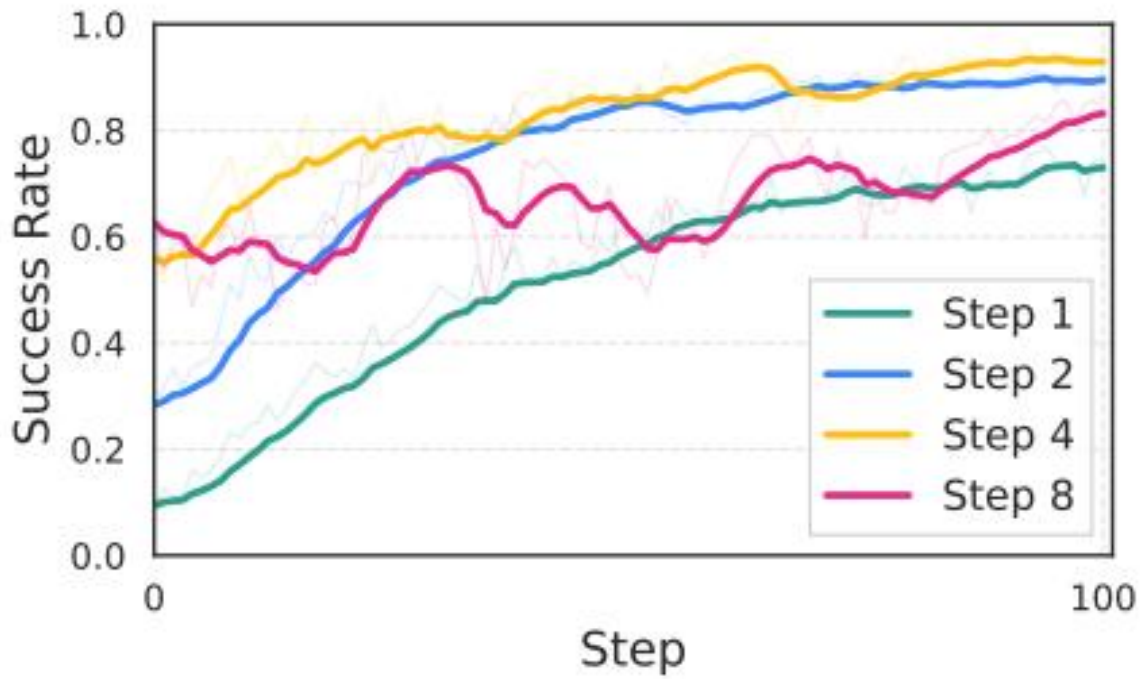


图 14: 去噪步数 (denoise step) 的消融实验: Train 结果。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

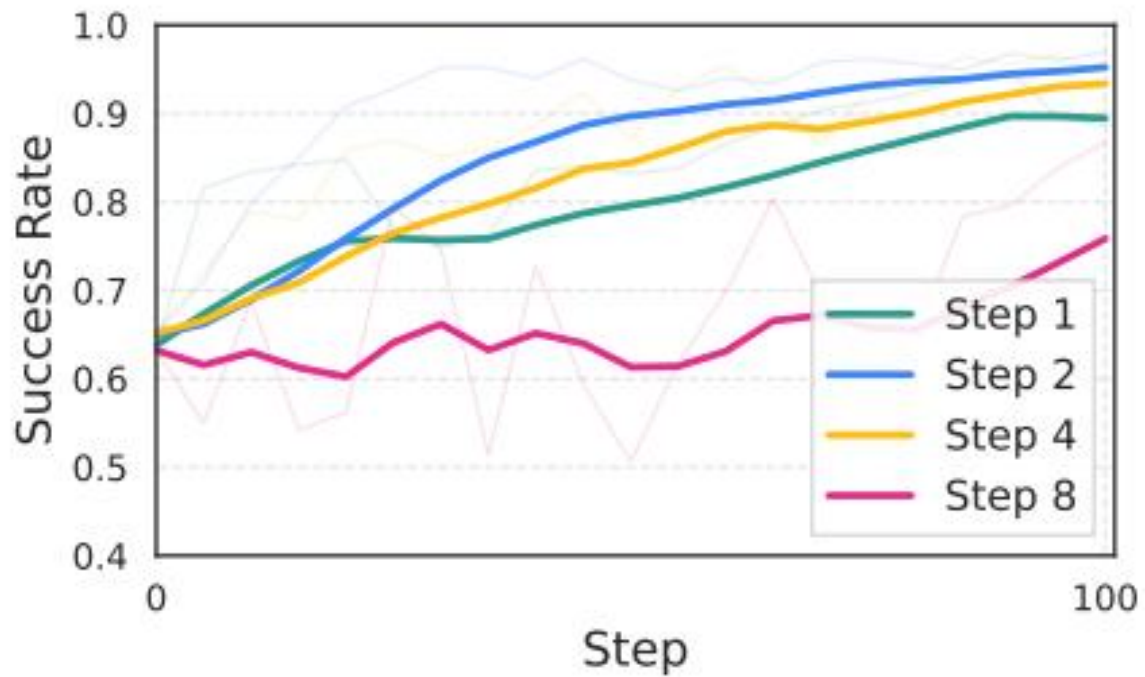


图 15: 去噪步数 (denoise step) 的消融实验: Eval 结果。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

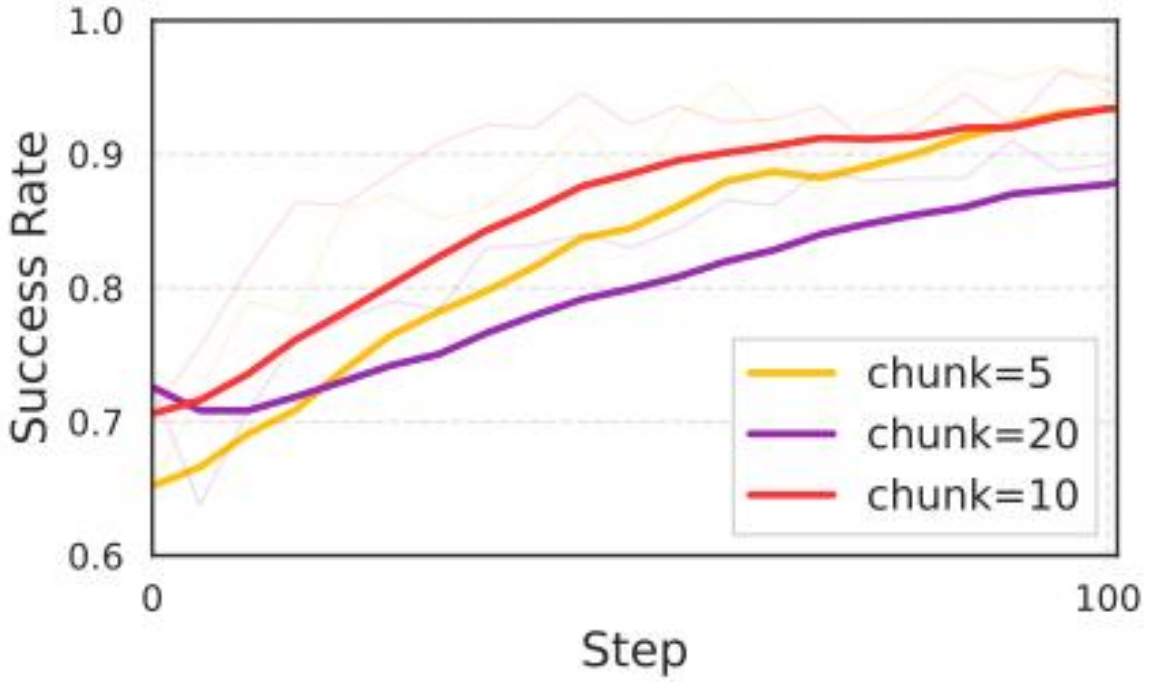


图 16: chunk size 的消融实验: Eval 结果。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

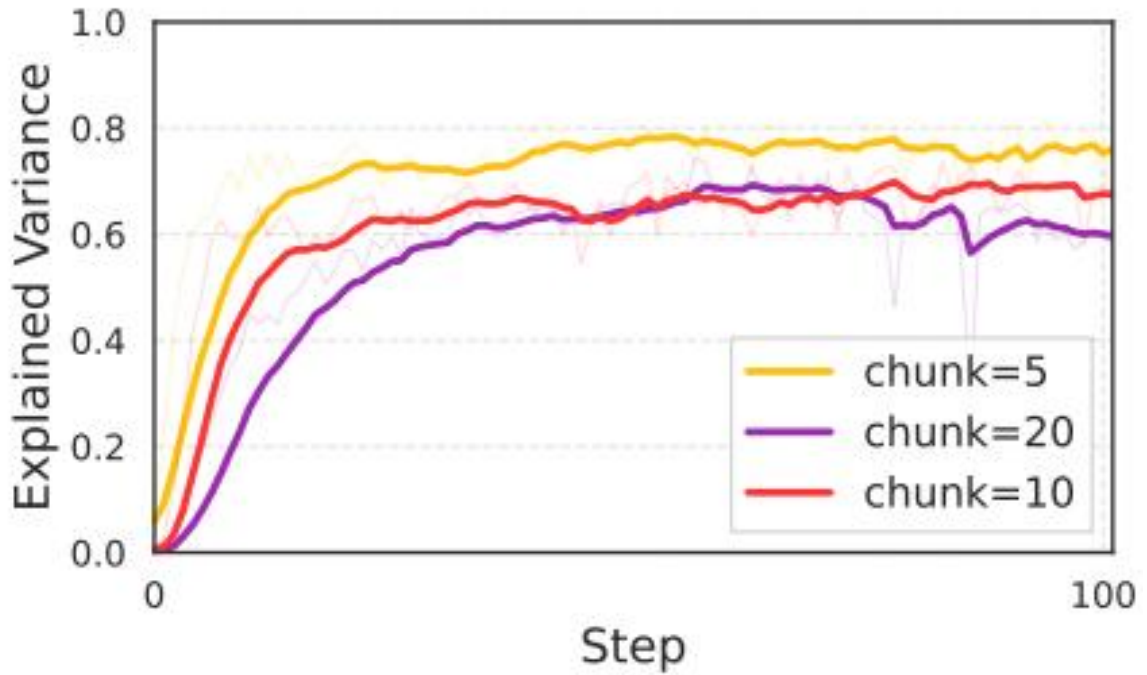


图 17: chunk size 的消融实验: Explained Variance。实验基于 LIBERO-Spatial 上的 Flow-SDE π_0 。

8.9 5.4. 超参数

基于带有 π_0 模型的 Flow-SDE，我们在 LIBERO-Spatial 基准上考察了噪声水平、denoise step 以及 action chunk 的影响。本文将训练阶段定义为 policy 生成随机动作以进行探索的阶段，而评估阶段则生成确定性动作。表 2 给出了 SFT baseline，以及经过 100 个训练步后的 RL 微调模型，在训练与评估两种设置下的成功率（success rate）。

噪声水平 (Noise Level) Flow-SDE 中的噪声水平 a 定义见式 (8)，用于控制去噪过程中注入噪声的幅度。如表 2 所示，由于 SFT baseline 依赖确定性的 ODE 采样，其评估性能在不同噪声水平下保持一致。相对地，其训练性能会随着噪声增大而明显下降。这一直观上是合理的，因为更高的噪声水平会扰动 flow 轨迹，从而导致对边缘动作分布的不准确估计。

将这一分析扩展到 RL 微调阶段，可以看到一个关键权衡：较低的噪声水平虽然能够减轻探索带来的性能退化，但同时也会限制 RL 进一步优化的空间。图 9 从经验上支持了这一点：当使用极低噪声 ($a = 0.2$) 进行训练时，训练会变得不稳定，并表现为显著更高的 clip fraction。我们将这种不稳定性归因于低噪声区间对应的梯度幅值显著更大。

Denoise Step denoise step K 决定了动作生成时的离散化步数，对于控制式 (8) 中 ODE 到 SDE 转换的保真度至关重要。从表 2 可以看出，尽管所有配置在初始时的 eval 性能相近，但当 $K = 1$ 时，训练成功率会急剧下降，这表明 ODE 到 SDE 的离散化误差非常显著。

不过，更大的 K 也并非一定更优。图 10 表明，增大 K 会带来一个明确的权衡：一方面，它能够提升 rollout 性能；另一方面，由于去噪步骤序列变长，也会同步增加训练复杂度与计算开销。

Action Chunk action chunk 表示 policy 在单次观测下连续执行的动作数量。我们对 action chunk 大小分别取 5、10 和 20 进行了消融，结果见图 11。

尽管更大的 chunk size 能带来边际上的性能提升，但它本质上会降低 policy 与环境交互的频率，并削弱奖励归因的精细程度。这些限制会导致 advantage 估计的可靠性下降，具体表现为 explained variance 变低。因此，尽管增大 chunk size 可能带来更强的 SFT baseline，它却会反过来限制后续由 RL 驱动的进一步提升上限。

9 6. Conclusion

我们提出了 π_{RL} ，这是一个可将基于 flow 的 VLA—— π_0 与 $\pi_{0.5}$ ——通过在线 RL 算法进行微调 (fine-tuning) 的框架。针对 flow matching 中对数似然难以计算这一核心挑战，我们提出了 Flow-Noise 与 Flow-SDE 两种解决方案。我们在具有挑战性的基准任务上开展了大量实验，结果表明，相较于 SFT baseline， π_{RL} 能够带来显著的性能提升。

局限性。 由于在线 RL 的样本效率较低，该框架目前仍依赖 sim-to-real 部署。未来，我们希望进一步发展更高效的算法，以支持在真实世界中直接开展 RL 训练。

10 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。

10.1 A. 附录

本附录补充介绍 π_{RL} 的更多技术细节与实验结果。内容组织如下：

- 附录 B：实验设置。
- 附录 C：分布内 (In-Distribution) RL 训练。

- 附录 D: 分布外 (Out-of-Distribution) RL 评估。
- 附录 E: 案例研究: 单任务 RL 训练。
- 附录 F: 消融实验细节。
- 附录 G: 大规模 RL 训练的经验与观察。
- 附录 H: 面向 GR00T N1.5 的 RL。
- 附录 I: 局限性与未来工作。
- 附录 J: 训练超参数。

10.2 B.2. 实现细节

考虑到预训练模型在任务特定 benchmark 上往往难以实现良好的泛化,我们首先基于专家 demonstration 进行 SFT。在 SFT 阶段,我们遵循官方设置,对整个 3.3B 模型进行微调 (fine-tuning)。在随后的 RL 阶段,出于 GPU 显存效率的考虑,并结合 RL4VLA 的结论——即 RL 对动作泛化的贡献更为显著——我们冻结 VLM 参数,仅对 300M 的 action expert 模型进行微调。

整个框架基于 RLinf (Yu et al., 2025) 代码库实现。我们采用共享、同机放置的 GPU 分配策略,将 environment、rollout model 和 actor model 部署在同一张 GPU 上,并以串行方式执行。

在模型配置方面,我们遵循 openpi (Black et al., 2024; Intelligence et al., 2025) 提供的官方设置。在该设置下, π_0 以图像、语言和 proprioceptive state 作为输入,而 $\pi_{0.5}$ 则显著地去除了状态信息²。我们的实验在 8 张 NVIDIA H100 80GB GPU 上完成,详细的训练超参数见附录第 J 节。

10.3 B. Experimental Setup

10.4 B.1. Benchmarks.

为严格评估本文框架的性能与泛化能力,我们在四个具有代表性的 benchmark 上进行实验。这些 benchmark 分别从不同侧面考察机器人的能力。

- **LIBERO** 侧重任务中的组合变化与知识迁移。我们在四个任务套件上进行评测: Spatial、Object、Goal 和 Long。该 benchmark 主要考察模型能否将基础技能迁移到不同的物体排列变化,以及长时程任务中。
- **ManiSkill** 强调在大规模环境多样性下的感知鲁棒性与执行鲁棒性。我们遵循 RL4VLA (Liu et al., 2025a) 的设置,构建了一个包含 4,352 种独特 pick-and-place 组合的 benchmark。该 benchmark 要求 policy 在大范围物体与容器分布上始终稳定维持物理交互。
- **CALVIN** 评估长时程序列推理能力以及视觉-语言 grounding 能力。它要求模型在持续存在的环境中执行由 5 个随机子任务构成的任务链,因此需要模型能够与语言指令实现准确对齐。
- **MetaWorld** 衡量技能覆盖广度与多任务通用性。它要求单一 policy 掌握 50 种语义上不同的 manipulation 基元,任务范围从简单的 reaching 到复杂的工具使用。

10.5 C. In-Distribution RL Training

本节在多个 benchmark 上评估所提框架的有效性,以验证其在 ID 设定下的鲁棒性与优越性能。

10.6 C.1. LIBERO Benchmark

SFT 过程。 LIBERO benchmark 包含四个任务套件 (task suite)，每个套件由 10 个不同的子任务组成。为了在 LIBERO 上开展 few-shot SFT，每个子任务在四个任务套件中至少需要 40 条 expert demonstration 轨迹，才能保证其成功率 (success rate) 为正，从而为后续 RL 阶段提供有效的正向优化信号。

我们按照官方训练配置执行 few-shot SFT。对于 π_0 模型，我们从总计 1,692 条 demonstration³ 中采样 58 条轨迹进行微调 (fine-tuning)，并将其作为后续在 Spatial、Object 和 Goal 任务套件上进行 RL 训练的初始 checkpoint。对于 Long 任务套件，由于其更具挑战性且具有 long-horizon 特征，我们使用了更大的 208 条轨迹集合。相比之下， $\pi_{0.5}$ 模型受益于更优的预训练 checkpoint 和训练配置，因此仅使用 40 条轨迹，便可为所有任务套件提供统一的 few-shot SFT checkpoint。

RL 过程。 在 RL 阶段，VLA 模型接收的多模态输入状态包括：agent-view 与 wrist-view 图像、自然语言指令、机器人 end-effector 位姿以及 gripper 状态。模型输出 action 与 LIBERO 环境交互；当任务成功完成时，环境返回二值奖励 1，否则返回 0。

表 3: LIBERO benchmark 上的评测结果, 评价指标为成功率 (%)。

Model	Spatial	Object	Goal	Long	Avg.	Δ Avg.
# Full Dataset SFT						
Octo	78.9	85.7	84.6	51.1	75.1	
OpenVLA	84.7	88.4	79.2	53.7	76.5	
π_{fast}	96.4	96.8	88.6	60.2	85.5	
OpenVLA-OFT	91.6	95.3	90.6	86.5	91.0	
π_0	96.8	98.8	95.8	85.2	94.2	
$\pi_{0.5}$	98.8	98.2	98.0	92.4	96.9	
# Few-shot SFT + RL						
SFT π_0	65.3	64.4	49.8	51.2	57.6	
Flow-SDE	98.4	99.4	96.2	90.2	96.1	+38.5
Flow-Noise	99.0	99.2	98.2	93.8	97.6	+40.0
# Few-shot SFT + RL ($\pi_{0.5}$)						
SFT $\pi_{0.5}$	95.4	84.6	43.9	77.1		
Flow-SDE	99.6	100	98.8	93.0	97.9	+20.8
Flow-Noise	99.6	100	99.6	94.0	98.3	+21.2

表 4: ManiSkill benchmark 上的评测结果；更具体的 OOD 结果见表 7。

Model	Method	IND		OOD				
		Avg.	Δ	Vision	Semantic	Execution	Avg.	Δ
π_0	SFT	38.4		32.6	8.4	13.2	18.1	
	Flow-SDE	78.8	+40.4	61.1	25.4	31.5	39.3	+21.2
	Flow-Noise	77.8	+39.4	63.4	23.1	24.2	36.9	+18.8
$\pi_{0.5}$	SFT	40.1		40.2	16.6	22.4	26.4	
	Flow-SDE	90.9	+50.8	68.0	34.5	45.4	49.3	+22.9
	Flow-Noise	89.7	+49.6	69.9	35.5	54.9	53.4	+27.0

10.7 C.1. LIBERO Benchmark (第 2 部分)

实验设置。 我们将 π_{RL} 的性能与多种 SOTA VLA 进行基准比较。这里的 π_{RL} 由少样本 SFT 的 π_0 和 $\pi_{0.5}$ 模型进一步结合 Flow-Noise 与 Flow-SDE 微调得到。对比方法包括在完整 LIBERO 数据集上训练的 Octo、OpenVLA、OpenVLA-OFT、 π_{fast} (Pertsch et al., 2025)、 π_0 以及 $\pi_{0.5}$ 。我们在 LIBERO 的四个任务套件上开展实验，并将全部 500 个初始状态上的成功率 (success rate) 作为性能指标进行汇报，其中包含 10 个子任务，每个子任务对应 50 个状态。

结果分析。 对于少样本 π_0 模型，SFT baseline 表现较差，平均成功率仅为 57.6%，说明该模型在 demonstration 数据有限时较难学到有效 policy。相比之下， π_{RL} 显著提升了性能：采用 Flow-SDE 和 Flow-Noise 时，成功率分别达到 96.1% 和 97.6%，并超过了基于完整数据集训练的 π_0 SFT baseline (94.2%)。

对于少样本 $\pi_{0.5}$ ，SFT baseline 的平均性能尚可，达到 77.1%，但在更具挑战性的 Long 任务套件上表现明显不足，得分仅为 43.9%。我们提出的 π_{RL} 有效弥补了这一缺陷，将 Long 任务的成功率从 43.9% 提升至 94.0%，增幅达到 50.1%。值得注意的是，尽管在 SFT 阶段仅使用了单条轨迹， π_{RL} 的最终性能仍达到 98.3%，超过了基于完整数据集训练的 SFT 模型的 96.9%。

10.8 C.2. ManiSkill Benchmark

SFT 过程。 在 ManiSkill benchmark 中，policy 需要从 16 类物体中抓取目标，并将其放置到 17 个容器之一中；这些任务分布在 16 个不同的桌面场景上，总计构成 4,352 种不同的任务组合。由于该设定具有较高复杂度，SFT 数据集由 16,384 条 episode 组成，使用 MPLib motion planning suite 合成生成 [?]。此外，为了强化模型对“动作完成”这一概念的学习，在每条轨迹末尾额外附加了 15 帧。

表 5: CALVIN benchmark (Scene D) 上的评测结果, 报告长度为 1 到 5 的任务序列的平均完成子任务数与成功率 (success rate)。

Methods		CALVIN-D						
		Len-1	Len-2	Len-3	Len-4	Len-5	Avg.	Δ Avg.
π_0	SFT	94.7	84.9	74.3	65.2	57.5	3.766	
	Flow-SDE	96.4	88.0	77.5	70.8	61.7	3.944	+0.178
	Flow-Noise	96.9	88.8	78.0	68.3	59.9	3.919	+0.153
$\pi_{0.5}$	SFT	92.7	84.3	76.7	68.8	61.3	3.838	
	Flow-SDE	99.7	98.2	95.8	91.0	87.0	4.717	+0.879
	Flow-Noise	99.6	97.6	93.9	89.6	84.5	4.652	+0.814

表 6: MetaWorld MT50 benchmark 上的评测结果。

Methods		MetaWorld-MT50					
		Easy	Medium	Hard	Very Hard	Avg.	Δ Avg.
	Diffusion Policy	23.1	10.7	1.9	6.1	10.5	
	TinyVLA	77.6	21.5	11.4	15.8	31.6	
	SmolVLA	87.1	51.8	70.0	64.0	68.2	—
π_0	SFT	77.9	51.8	53.3	20.0	50.8	
	Flow-SDE	92.1	74.6	61.7	84.0	78.1	+27.3
	Flow-Noise	91.1	81.8	78.3	92.0	85.8	+35.0
$\pi_{0.5}$	SFT	68.2	37.3	41.7	28.0	43.8	
	Flow-SDE	86.4	55.5	75.0	66.0	70.7	+26.9
	Flow-Noise	86.8	58.1	63.3	56.0	66.1	+22.3

RL 过程。 在 RL 阶段, VLA 模型接收第三人称 RGB 图像、简洁的语言指令以及当前关节 proprioception。环境提供结构化奖励信号: 当物体被正确放置时给予 1.0 的奖励; 当 gripper 与物体成功附着时, 额外给予 0.1 的辅助奖励。该设计旨在鼓励更稳定的操纵行为, 并抑制诸如冲动式抛掷这类

不期望的行为。

实验。 遵循 RL4VLA 的实验协议，我们在完整的 4,352 种任务组合上开展 RL 训练，并以这些任务上的整体成功率作为性能指标。

分析。 如表 4 所示， π_{RL} 在训练环境中显著提升了性能。具体而言， π_0 的成功率从 38.4% 提升至 77.8%，而 $\pi_{0.5}$ 则从 40.1% 提升至 90.9%。这些结果表明，在复杂设定下，RL 能够带来显著收益。

10.9 C.3. CALVIN Benchmark

SFT 过程。 我们在 CALVIN ABC 数据集⁴上进行 SFT。该数据集包含约 24 小时的非结构化 “play data”，覆盖三个不同环境（A、B 和 C）。数据集中包含超过 20,000 条带语言标注的轨迹，涉及 34 个不同的操纵任务。

RL 过程。 每个 episode 由 5 个随机采样的子任务顺序组成，要求在不重置环境的情况下依次完成。奖励信号定义在子任务层级：每当模型成功执行一个子任务时，获得一个稀疏的二值奖励 1.0；否则奖励为 0.0。

实验设置。 我们在 Scene D 中对 π_{RL} 的性能进行评估，共测试 1,000 个 episode。遵循标准的 CALVIN 评测协议，我们报告两个关键指标：(1) 不同任务序列长度下的成功率（success rate），即 Len-1 到 Len-5；(2) 每个 episode 中平均完成的子任务数，记为 Avg.。

分析。 如表 5 所示， π_{RI} 在长时程序列执行中带来了显著的性能提升，其中 $\pi_{0.5}$ 变体尤为突出。SFT 模型天然容易受到序列任务中误差累积的影响；即使是 $\pi_{0.5}$ ，在 Len-5 序列上的成功率也只有 61.3%。RL 能够有效缓解这一问题：Flow-SDE 的平均完成子任务数从 3.838 提升至 4.717，而其在 Len-5 上的成功率则大幅提升到 87.0%。

表 7: ManiSkill OOD 设定下的具体泛化评测结果。

Environment	Variation-Version-Type	π_0 -SFT	π_0 -RL Flow-SDE	π_0 -RL Flow-Noise	$\pi_{0.5}$ -SFT	$\pi_{0.5}$ -RL Flow-SDE	$\pi_{0.5}$ -RL Flow-Noise
In distribution	Main-v3-train	38.4	78.8	77.8	40.1	90.9	89.7
Visual-Language	Instruct-v1-test	30.1	64.6	66.5	46.6	77.0	85.7
	VisionImage-v1-test	38.3	68.8	71.7	46.2	78.8	83.1
	VisionTexture03-v1-test	35.1	66.0	66.8	36.7	69.6	75.0
	VisionTexture05-v1-test	31.0	55.8	60.5	32.7	58.0	62.2
	VisionWhole03-v1-test	35.4	62.4	69.0	40.1	69.6	71.6
	VisionWhole05-v1-test	28.5	49.0	53.9	30.7	55.0	57.0
Semantic Reasoning (object/receptacle confounders)	MultiCarrot-v1-test	7.8	28.2	23.0	16.7	36.8	38.2
	MultiCarrot-v1-train	12.5	36.5	31.8	28.2	49.5	50.1
	MultiPlate-v1-test	5.0	16.4	18.3	11.8	29.4	28.3
	MultiPlate-v1-train	7.3	20.5	19.6	9.7	22.3	25.4
Action Execution	PositionChangeTo-v1-test	9.6	17.4	10.9	13.5	36.2	54.7
	Position-v1-test	16.9	45.6	37.5	31.2	54.5	55.0

值得注意的是，随着序列长度增加，SFT 与 RL 之间的性能差距显著扩大。对于 $\pi_{0.5}$ ，Flow-SDE 在 Len-1 任务上相较 SFT 仅提升了 7.0%；但在最具挑战性的 Len-5 序列中，这一差距扩大到了 25.7%。

10.10 C.4. MetaWorld Benchmark

SFT 过程。我们在 π_0 和 $\pi_{0.5}$ 模型上, 使用官方数据集⁵ 进行 SFT。该数据集包含 50 种不同操纵任务上的 2500 条轨迹。

RL 过程。在 RL 过程中, VLA 模型接收多模态输入, 包括一张 RGB agentview 图像、语言指导、机器人的 end-effector 位置以及 gripper 状态。基于这些输入, 模型输出与环境交互的 action。环境随后返回稀疏奖励: 任务成功完成时奖励为 1, 否则为 0。

实验。我们将 π_{RL} 与 Diffusion Policy (Chi et al., 2025)、TinyVLA (Wen et al., 2025) 和 SmolVLA (Shukor et al., 2025) 进行基准比较。在性能评估时, 我们遵循 SmolVLA 的设置, 即根据任务难度, 将 50 个任务划分为 easy、medium、hard 和 very hard 四类。

分析。如表 6 所示, RL 微调 (fine-tuning) 显著提升了模型性能。其中, π_0 和 $\pi_{0.5}$ 的平均成功率 (success rate) 分别达到 85.8% 和 70.7%。相较于仅经过 SFT 的对应模型, 这一结果有明显提升; 同时也超过了 SmolVLA 的 baseline 结果 68.2%。这表明, RL 能够在多种不同类型的操纵任务上有效增强模型能力。

10.11 D. 分布外 RL 评估

尽管前述实验已经表明, RL 驱动的改进在 ID 域内十分显著, 但本节进一步评估其在 OOD 场景下的泛化能力。由于 LIBERO 缺乏专门用于 OOD 测试的接口, 我们采用 ManiSkill、CALVIN 和 MetaWorld 基准, 以考察这些由 RL 驱动带来的性能提升, 究竟体现为能够扩展到新环境中的真实技能习得, 还是仅仅利用了特定环境中的偏置。

10.12 D.1. ManiSkill

设置 遵循 RL4VLA, 我们在三类具有挑战性的 OOD 场景下评估模型的泛化能力: (1) **Vision**, 通过新的背景与纹理考察模型的视觉泛化; (2) **Semantics**, 利用未见过的物体、变化的指令, 以及额外物体或容器等干扰因素, 检验模型的语义理解能力; (3) **Execution**, 通过不同初始状态、未见过的机器人位姿以及动态扰动, 评估模型在执行层面的鲁棒性。

结果 从表 4 和表 7 所示的 OOD 场景结果可以看出, π_0 -SFT 模型在视觉信息上的泛化能力较强。这主要得益于其 VLM 所提供的稳健基础, 使其能够更好地应对视觉扰动。

然而, π_0 在语义相关任务上的性能下降非常明显。切换到 $\pi_{0.5}$ baseline 后, 这种退化有所缓解, 这很可能受益于预训练 $\pi_{0.5}$ 模型所具备的知识泛化能力。在动作执行方面, π_0 的性能降幅也大于 $\pi_{0.5}$ 。我们推测, 这种差异源于 π_0 将关节角状态作为输入, 从而在控制任务中产生了更严重的过拟合; 相比之下, $\pi_{0.5}$ 不使用这些输入, 因此避免了同等程度的性能退化。

表 8: 采用 Flow-Noise 方法时, π_0 与 $\pi_{0.5}$ 在 SIMPLER benchmark 上的评测结果。

Model		SIMPLER				
		Carrot	Eggplant	Spoon	Cube	Avg.
π_0	SFT	82.7	87.5	61.7	37.1	67.2
	+RL	95.7	96.7	91.6	63.0	86.7
	Δ	+13.0	+9.2	+29.9	+25.9	+19.5
$\pi_{0.5}$	SFT	70.6	91.9	43.5	31.0	59.2
	+RL	82.0	98.2	82.8	53.3	79.1
	Δ	+11.4	+6.3	+39.3	+22.3	+19.9



图 18: 经 RL 优化后的 policy 在真实世界部署中执行 pick-and-place 任务。

对于 RL 训练, 尽管其在 OOD 场景下带来的性能提升低于 IND 设置, 但相对提升幅度仍然具有显著可比性。正如表 4 所示, 对于 $\pi_{0.5}$ 模型, Flow-SDE 使 IND 成功率 (success rate) 提升了 126.7%, 而在 OOD 场景下也实现了 102.3% 的提升。这样的相对增益一致性表明, 由 RL 驱动的优化促进了更具泛化性的动作表征学习, 而不仅仅是对训练环境的过拟合, 因此在分布偏移下仍能保持有效性。

不过, 我们也观察到, Vision OOD 任务与 IND 域之间在相对提升上仍存在差距。具体而言, 对于 $\pi_{0.5}$, 其视觉泛化收益为 73.9%, 低于训练环境中观察到的 126.7% 提升。我们认为, 这一差异很可能是因为在 RL 阶段出于计算效率考虑冻结了 VLM backbone, 从而限制了模型将视觉 grounding 特征适配到新纹理和新背景的能力。

10.13 D.2. CALVIN

设置. 我们基于 CALVIN 中的 ABC $\rightarrow$ D 协议, 评估模型在环境与视觉层面的 OOD 鲁棒性。在该设定下, 模型在 ABC 环境上训练, 并以 zero-shot 方式在 Scene D 上进行评估。Scene D 在视觉纹理、光照条件和空间布局等方面都引入了显著的分布偏移, 因此能够有效检验智能体将已学技能迁移到陌生物理环境中的能力。

结果. 在相同的 D→D 训练设定下, 在 ABC 环境中经过 RL 微调 (fine-tuning) 的 policy 在 OOD 场景 D 上取得了 79.1% 的成功率 (success rate); 如图 5 所示, 相比 61.3% 的 SFT baseline 有明显提升。这一结果与 ManiSkill 中的发现一致, 表明在 ID 设定下获得的性能增益可以迁移到由视觉变化主导的 OOD 场景中。

10.14 D.3. MetaWorld

设置. 我们采用 MetaWorld 的 ML45 benchmark 评估任务层面的泛化能力。该设置包含 50 个不同的机器人操纵任务: 模型在其中 45 个基础任务上训练, 随后在 5 个保留的、训练期间未见过的任务上测试。因而, 这一评测要求模型将已学到的操纵原语泛化到全新的任务目标与工作空间配置中。

结果. 如图 5 所示, 尽管模型在 ID 域内的成功率 (success rate) 持续提升, 但 OOD 性能在整个训练过程中始终表现出明显振荡。这种不稳定性表明, 就当前形式而言, RL 仍难以实现稳定的跨类别泛化。

尽管如此, 在 RL 训练的全过程中, 模型依然能够保留其在 SFT 阶段获得的 OOD 能力。这一点相较于标准 SFT 具有显著优势: 后者往往会使模型在专家 demonstration 上过拟合, 并丧失更广泛的通用能力 (Li et al., 2025a)。与标准 SFT 不同, RL 能够在提升 ID 性能的同时保留通用知识, 这说明 RL 提供了一种更均衡的优化方式, 从而在避免灾难性遗忘的同时维持 OOD 鲁棒性。

表 9: 在 LIBERO 上, 结合 Flow-SDE 时 PPO 与 GRPO 的比较。

Model		LIBERO					
		Spatial	Object	Goal	Long	Avg.	Δ Avg.
π_0	SFT	65.3	64.4	49.8	51.2	57.6	
	+GRPO	97.8	97.8	83.2	81.4	90.0	+32.4
	+PPO	98.4	99.4	96.2	90.2	96.0	+38.4
$\pi_{0.5}$	SFT	84.6	95.4	84.6	43.9	77.1	
	+GRPO	97.4	99.8	91.2	77.6	91.5	+14.4
	+PPO	99.6	100	98.8	93.0	97.9	+20.8

10.15 D.4. 总结

综上, 我们的 OOD 评测表明: RL 能够提升与训练任务相近任务上的性能, 但在泛化到全新任务目标时仍然效果有限。

具体而言, RL 训练能够有效增强模型对低层次变化的鲁棒性, 例如 ManiSkill 和 CALVIN 中出现的视觉分布偏移与执行偏移。这说明模型学到的是更具泛化性的 action representation, 而不仅仅

是对训练环境的过拟合。

在 MetaWorld 所考察的高层次泛化方面，模型能够较好保留 SFT 阶段获得的 OOD 技能，这表明 RL 避免了标准模仿学习中常见的灾难性遗忘与过拟合问题。然而，如何将这些性能增益进一步迁移到完全新的任务目标上，仍然是一个重要挑战。

10.16 E. 案例研究：单任务 RL 训练

前面的实验主要关注模型在多任务基准上的整体表现；本节进一步考察单任务场景，即在相对静态的环境中训练 VLA，使其掌握某一特定任务。具体而言，我们在 SIMPLER (Li et al., 2024) 以及一个 Real2Sim2Real 环境中评估了所提出的方法。

10.17 E.1. SIMPLER

设置。 在 SIMPLER 中，实验配置采用一台 8-DoF 的 WidowX-250S 机械臂，并在四个标准任务上进行评估：

1. Spoon: 将勺子放到布上。
2. Carrot: 将胡萝卜放到盘子上。
3. Eggplant: 将茄子放入篮子中。
4. Cube: 堆叠一个立方体。

在 SFT 阶段,我们使用了一个经过筛选的数据集,其中每个任务均使用 144 条 demonstration episode 进行训练。

分析。 如表 8 所示, π_{RL} 将模型 π_0 的平均成功率 (success rate) 从 67.2% 提升到 86.7%, 其中有三个任务 (carrot、eggplant 和 spoon) 的成功率超过了 90%。

10.18 E.2. Real2Sim2Real

尽管 SIMPLER benchmark 展示了仿真性能与真实世界性能之间的预测相关性，但显著的视觉域偏移问题仍有待解决。为此，我们采用近期的 Real2Sim2Real 方法 [?, ?]，基于 ManiSkill 构建高保真仿真环境以模拟刚体动力学，并结合 Gaussian Splatting [?] 实现照片级真实感渲染。

实验设置 我们的硬件平台由一台 Franka Panda 机械臂和一台 Intel RealSense D435 相机构成，其中相机作为主要视觉传感器。我们通过手动标定的方式，将仿真视角与真实相机视角对齐，从而同步二者的外参矩阵。如图 1 所示，仿真器中的真实感纹理与颜色分布与物理环境高度一致，从而有效减小了从仿真到现实的视觉域偏移。

实验结果 遵循 ManiSkill benchmark 所采用的实验协议，我们首先利用运动规划器采集 20 条 expert trajectory，用于 few-shot SFT；随后在此基础上进一步进行 RL 优化，共训练 100 次迭代。最终，我们将经过 RL 微调 (fine-tuning) 的 policy 以 zero-shot 方式部署到真实环境中。实验结果表明，SFT baseline 无法完成该任务，而经 RL 优化后的 policy 可达到 40% 的成功率 (success rate)。一个具有代表性的成功执行案例见图 12。

10.19 F. 消融实验细节

10.20 F.1. RL algorithms

鉴于 PPO 在 LIBERO benchmark 上带来了显著的性能提升，我们进一步考察了 GRPO (Shao et al., 2024) 的有效性。GRPO 是另一种广泛使用的 policy gradient 方法，也已被应用于 VLA+RL 训练 (Li et al., 2025a)。我们在 π_0 和 $\pi_{0.5}$ 两个模型上对 PPO 与 GRPO 的性能进行了比较，结果见 Tab. 9。

结论. 总体而言，我们的结果揭示了一个关键权衡：针对 rollout 成功率专门设计的参数设置，可能会对训练稳定性产生负面影响，从而限制 RL 的性能上限。因此，需要仔细调节参数，以在高质量 rollout 与稳定的 policy 收敛之间实现协同。

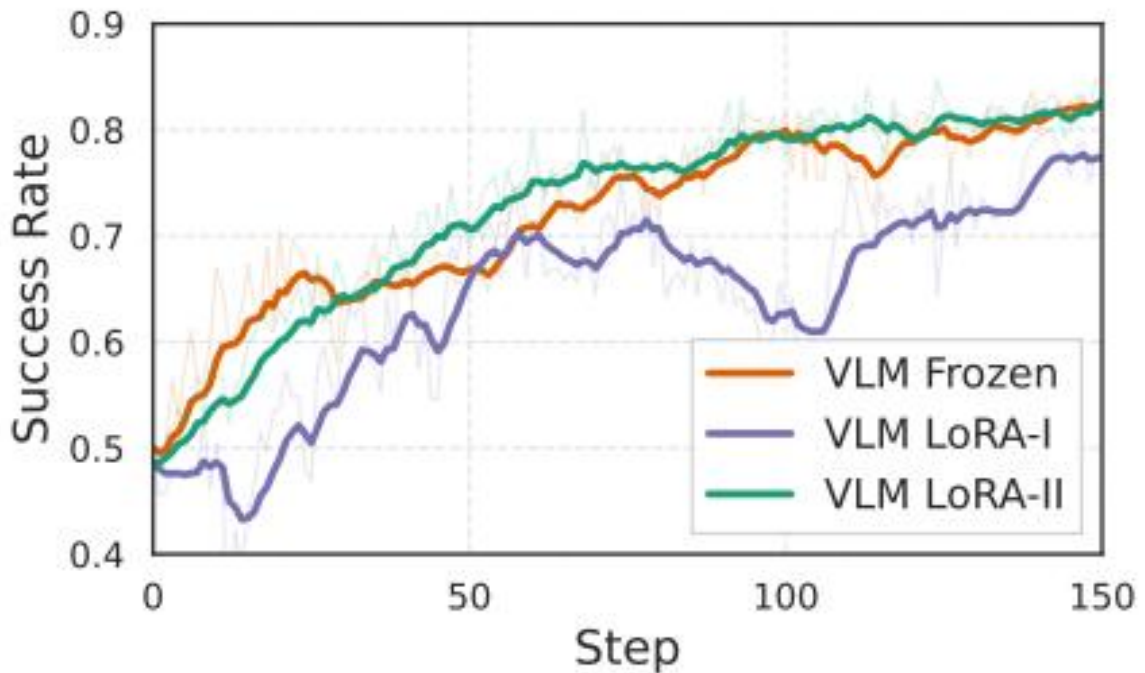


图 19: VLM 在 RL 过程中的有效性消融实验：(a) Eval。

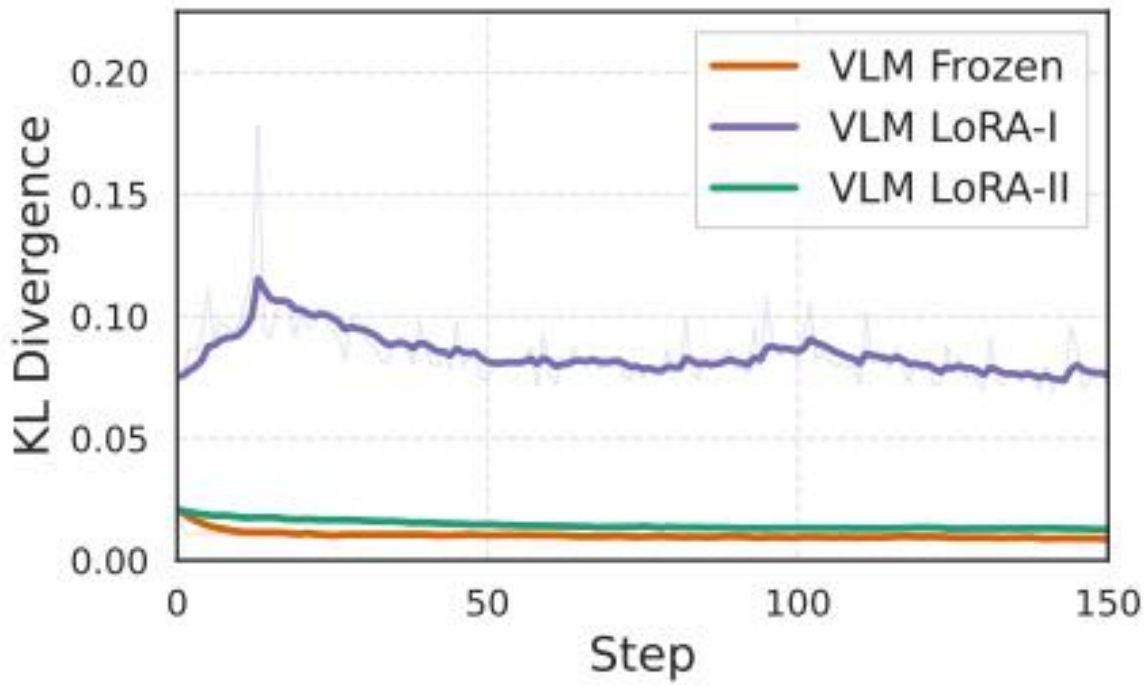


图 20: VLM 在 RL 过程中的有效性消融实验: (b) KL Divergence。

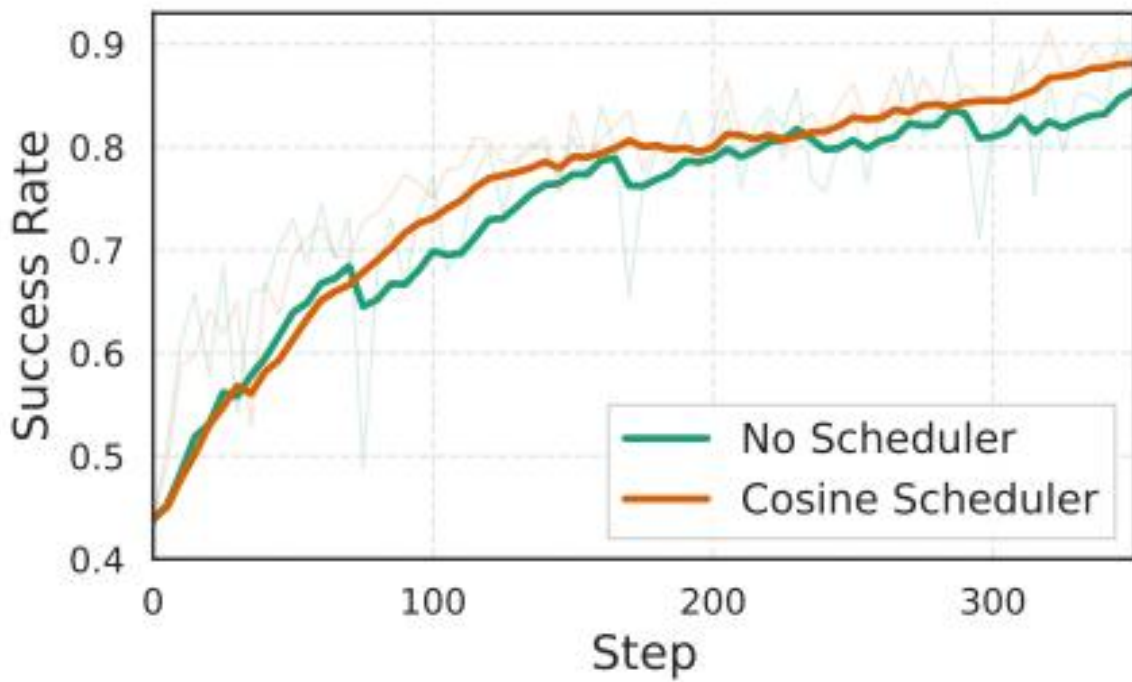


图 21: 学习率调度器的消融实验: (a) Eval。

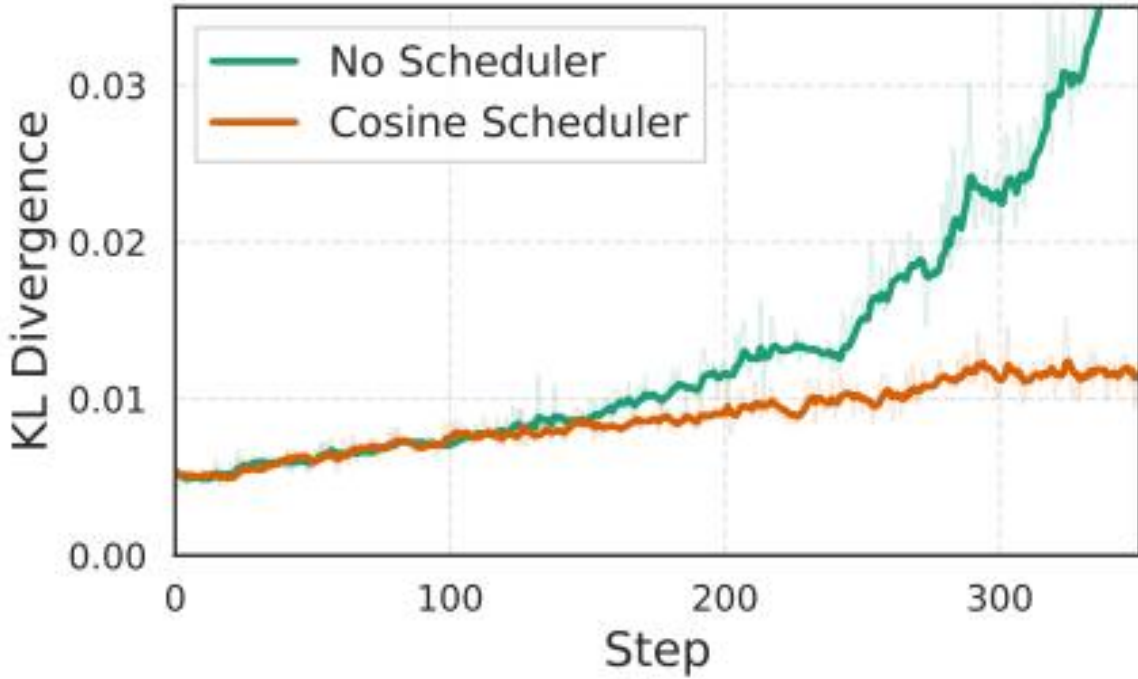


图 22: 学习率调度器的消融实验: (b) KL Divergence。

10.21 F.2. VLM 微调 (fine-tuning) 分析

在此前的实验中, VLM 保持冻结, RL 过程中仅优化 action expert。本小节旨在分析 VLM 在 RL 中所起的作用。具体而言, 我们对 VLM 采用 Low-Rank Adaptation (LoRA) [?], 从而能够与 action expert 联合优化。我们将 LoRA 的 rank 设为 $r = 32$, 缩放参数设为 $\alpha = 32$, 同时 action expert 保持完全可训练。

我们在 LIBERO-Long benchmark 上, 使用带有 Flow-SDE 的 π 模型开展实验, 并比较以下三种配置:

- **VLM Frozen:** 学习率为 $5e^{-6}$, 每个 epoch 更新 4 次。
- **VLM LoRA-I:** 学习率为 $5e^{-6}$, 每个 epoch 更新 4 次。
- **VLM LoRA-II:** 学习率为 $1e^{-6}$, 每个 epoch 更新 2 次。

如图 13 所示, VLM LoRA-II 的学习轨迹与 VLM Frozen baseline 基本相当。这一实证结果带来两点关键结论:

1. 在 LIBERO benchmark 上, 对 VLM 进行微调的收益并不明显。我们推测, 其性能增益有限的原因在于 LIBERO 中场景变化较少, 预训练 VLM 学到的表征已经足够稳健。
2. 将 VLM 与 action expert 一同微调时, 需要采用更保守的优化配置, 才能保证训练稳定性。

10.22 G. 大规模 RL 训练的经验洞见

本节总结我们在 RL 训练过程中获得的一些经验性观察。

超参数。 根据第 5.4 节中的超参数消融结果, 初始 SFT checkpoint 在训练阶段与评估阶段之间的性能差异值得重点关注。如果这种差异较大, 我们建议减小噪声幅度, 或者增加 denoising steps 的数

量，以缓解执行方式从确定性切换到随机性时带来的性能损失。进一步地，如前所述，较低的噪声水平会产生更大的梯度，因此需要配合更小的学习率来维持训练稳定性。

我们还观察到，当训练性能持续提升、而评估性能出现振荡时，增加 denoising steps 往往能够缓解这一问题。这一现象的原因在于：确定性动作生成过程与随机性动作生成过程之间的动作分布偏差会减小。对于 action chunk，我们的经验结果表明，长时域任务通常受益于更大的 chunk size。例如，在 LIBERO-Long 上我们将 chunk size 设为 10，而在其他子任务上设为 5。

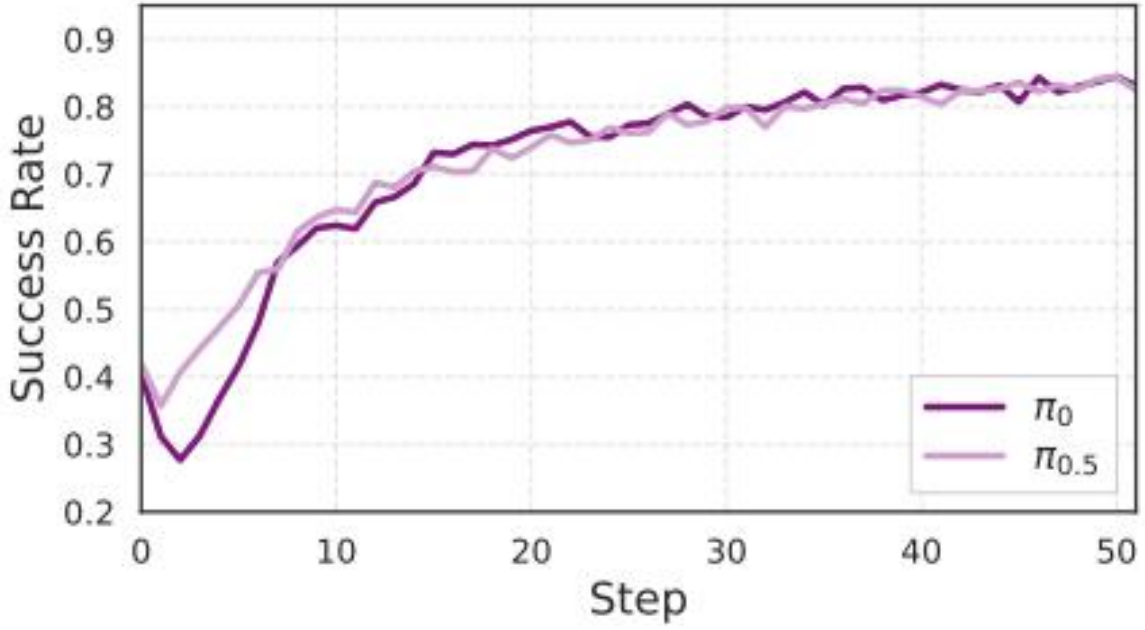


图 23: ManiSkill 中的训练曲线: (a) 评估性能 (Eval)。

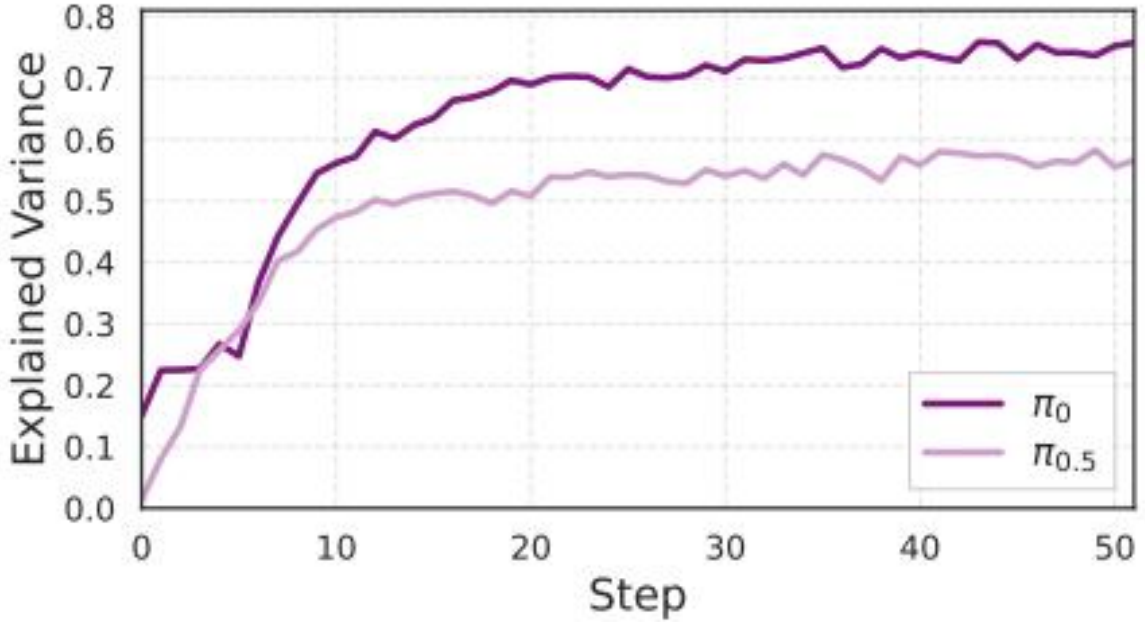


图 24: ManiSkill 中的训练曲线: (b) 解释方差 (Explained Variance)。

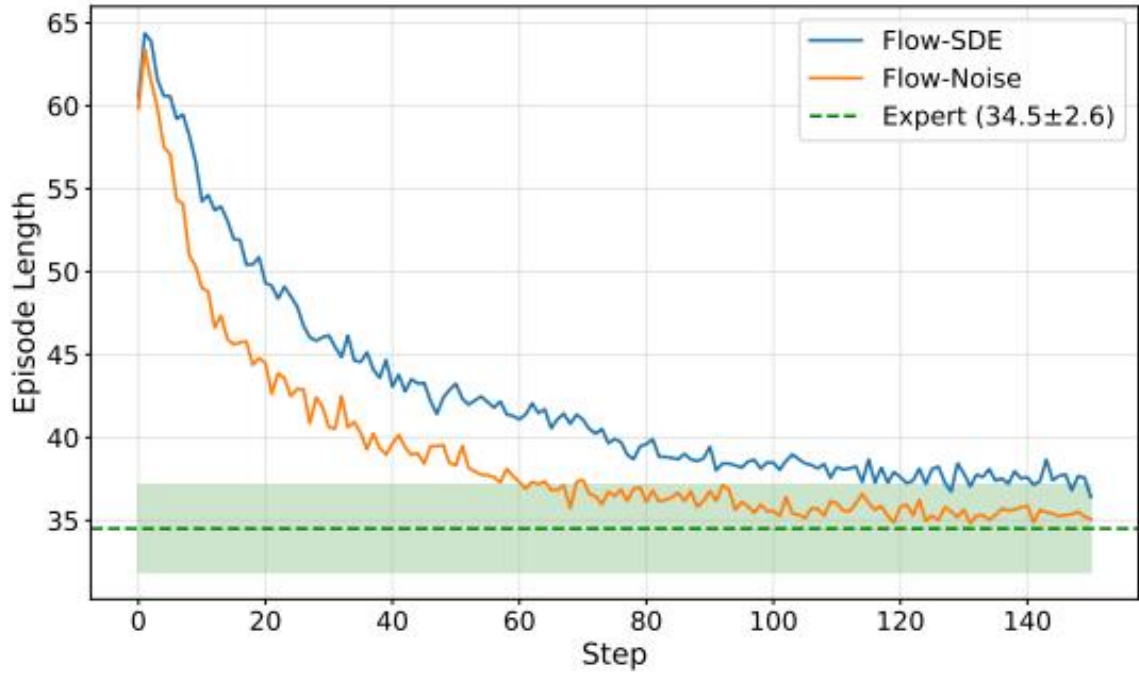


图 25: ManiSkill 中 $\pi_{0.5}$ 的 RL 训练 episode length。

训练。 在 LIBERO-Long 基准上的 $\pi_{0.5}$ 实验中，我们观察到 Kullback–Leibler (KL) 散度指标在整个训练过程中持续上升，这可能导致训练不稳定。为缓解这一问题，我们引入了带余弦退火 (cosine annealing) 的学习率调度器。正如图 14 所示，该调度器能够有效抑制 KL 散度持续增大，从而稳定训练过程。

Critic。 在 ManiSkill 实验中，我们发现，无论是 π_0 还是 $\pi_{0.5}$ ，其 policy 的评估性能都会先出现一次下降，随后再逐步提升，如图 15 所示。我们将这种短暂的性能退化归因于 critic 在预热阶段提供

了不够准确的学习信号。随后评估性能的提升，则与 critic 的 value 估计逐渐稳定直接相关，这一点也可以从 explained variance 的上升中得到验证。

时间效率。 我们还研究了：在物理仿真器中进行 RL rollout，如何帮助 policy 形成接近 expert 水平的的时间效率。具体而言，我们先分析了用于 SFT 的 expert 运动规划数据，随后在 ManiSkill 中跟踪了 $\pi_{0.5}$ 模型进行 RL 训练时的平均 episode length。图 16 表明，由 SFT 初始化得到的 policy 由于执行误差，episode 往往显著更长；相比之下， $\pi_{0.5}$ 在经过 RL 训练后，其 episode length 会收敛到 expert 的范围内，体现出时间效率的显著提升。

我们认为，这种收敛主要源于两个因素：

1. RL 提升了 policy 的纠错能力，使其能够从执行失败中恢复；
2. 我们采用的 partial reset 机制通过折扣奖励激励时间效率，因为更快完成任务意味着智能体能够在每次更新周期内触发更多次 reset，并累积更高的总奖励。

10.23 用于 GR00T N1.5 的强化学习 (RL)

10.24 H.1. Setup

GR00T N1.5. 我们基于 GR00T N1.5 模型 (Bjorck et al., 2025) 开展了额外实验。该模型是面向通用 humanoid 机器人推理与操纵的 foundation model。其架构将 Eagle 2.5 VLM (Chen et al., 2025) 与 Diffusion Transformer head (Peebles & Xie, 2023) 相结合：前者针对空间 grounding 与物理推理进行了优化，后者用于动作去噪 (action denoising)。该模型还通过专门设计的 head 支持多 embodiment 兼容，可适配带灵巧手或 gripper 的 humanoid，以及单臂操纵器等不同构型。

对于 critic 的实现，我们将 critic network 直接与 action head 集成，从而在整个去噪轨迹上估计 value function。完整框架如图 17 所示。

Benchmark. 我们在 LIBERO 的四组操纵任务上评估 GR00T 的性能，包括 Spatial、Object、Goal 和 Long。

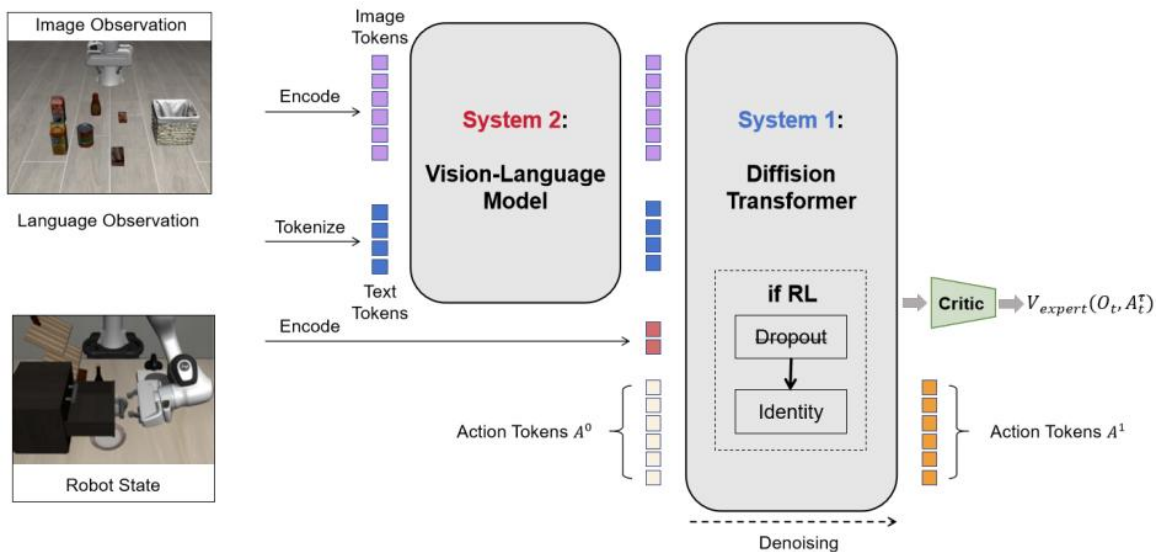


图 26: GR00T-N1.5 的架构示意图。

表 10: 在 LIBERO 上使用带 Flow-SDE 的 PPO 对 GR00T 进行微调 (fine-tuning) 的结果。

Model		LIBERO					
		Spatial	Object	Goal	Long	Avg.	Δ Avg.
GR00T	SFT	41.4	58.6	48.2	61.9	52.5	
	+PPO	92.5	96.2	84.3	86.6	89.9	+37.4

Implementation Details. 与 π_0 的实现类似, 我们首先在专家 demonstration 上进行 SFT。SFT 阶段遵循官方设置, 对整个模型进行微调 (fine-tuning); 随后在 RL 阶段, 我们仅微调 action expert model, 并保持 vision-language model 的参数固定。

我们在 RL pipeline 中做出了一项关键的方法改进: 将 expert model 中的 dropout layer 替换为 identity layer。众所周知, dropout 会在在线 RL 训练中引入不稳定性。具体而言, 它会对有效 policy 施加非确定性扰动, 使标准的概率比更新从

$$\rho_t(\theta) = \frac{\pi_{\theta_{\text{new}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (16)$$

变为一种高度不稳定的形式:

$$\rho_t(\theta) = \frac{\pi_{\alpha_{\text{new}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \quad (17)$$

其中, $\rho_t(\theta)$ 表示概率比, α_{new} 表示在随机 dropout mask 作用下、更新后的 policy 状态。此类结构性随机性再叠加逐步的 policy 更新, 会严重破坏训练收敛性。训练超参数与 π_0 所使用的设置完全一致, 主要 RL 算法采用 Flow-SDE。

10.25 H.2. 结果

结果汇总见表 10。对于 few-shot 模型, SFT baseline 的成功率 (success rate) 仅为 52.5%, 这表明仅依赖稀疏 demonstration 时, 其泛化能力较为有限。相比之下, 基于 RL 的 Flow-SDE 将性能显著提升至 89.9%。这些结果均在默认的 π_0 配置下取得, 说明我们的方法具有良好的跨架构适用性。尽管针对具体任务进行专门调优还有望进一步提升性能, 但这类优化留待未来工作展开。

11 I. Limitations and Future Work

11.1 噪声注入

我们当前采用的噪声注入策略在 ODE 到 SDE 的转换过程中会出现性能下降。Flow-CPS (Wang & Yu, 2025) 将这一损失归因于数值误差, 并提出了一种改进的、保持系数不变的采样方法。在我们的实验中, 也尝试了这一配置。与超参数消融实验的结论一致, 实验结果表明: 尽管该配置能够缓解 ODE-SDE 转换中的精度误差, 但对 RL 性能的提升较为有限。尽管如此, 我们认为, 改进噪声注入策

略仍具有重要潜力，尤其是在将 ODE 形式转换为 SDE 形式的同时，尽可能保持 action distribution 不受扰动。

11.2 训练加速

在 Flow-SDE 中，我们当前对混合 ODE-SDE rollout 的实现仍较为简单。具体来说，它会随机选择一个 denoising step 作为 SDE step，而其余步骤均保持为 ODE step。我们认为，未来若能结合 flow-based 图像生成加速方面的进展 (Li et al., 2025b; He et al., 2025; Liu et al., 2025b; Li et al., 2025c)，进一步研究混合 ODE-SDE rollout，有望继续提升 Flow-SDE，在加快训练速度的同时获得更好的性能。

11.3 泛化能力

Maniskill 的 OOD 测试表明，SFT 与 RL 模型的语义泛化能力仍然有限。为解决这一问题，未来工作将利用 RL，通过在更加多样的任务分布和更丰富的语言指令上进行训练来提升模型鲁棒性，从而增强跨任务适应能力。

11.4 J. 训练超参数（第 1 部分）

我们记录了在各个 benchmark 上训练 π_0 和 $\pi_{0.5}$ 时所使用的训练超参数，并将其汇总于表 11 至表 13 中。

表 11: LIBERO 上的超参数设置

参数	算法与任务							
	π_0				$\pi_{0.5}$			
	Spatial	Object	Goal	Long	Spatial	Object	Goal	Long
训练轮数 (Train epochs)	500	500	500	500	500	500	500	500
全局 batch size	2048	2048	2048	2048	2048	2048	2048	2048
更新轮数 (Update epochs)	4	4	4	4	1	1	3	4
Actor 学习率	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6
Critic 学习率	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
Scheduler	False	False	False	False	False	False	False	True
奖励折扣率 γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95
裁剪比率 ϵ	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
交互步数 (Interaction steps)	240	240	320	480	240	320	320	480
并行环境数	64	64	64	64	64	64	64	64
Rollout 轮数	8	8	8	8	8	8	8	8
动作预测 horizon H	50	50	50	50	10	10	10	10
动作重规划 horizon H'	5	5	5	10	5	5	5	10
去噪步数	4	4	4	4	3	5	5	5
噪声水平 σ (Flow-SDE)	0.5	0.5	0.5	0.5	0.5	0.3	0.3	0.5
最大 log-var (Flow-Noise)	0.16	0.16	0.16	0.16	0.10	0.10	0.10	0.10
最小 log-var (Flow-Noise)	0.08	0.08	0.08	0.08	0.04	0.04	0.04	0.04
熵奖励 (Flow-Noise)	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005

11.5 训练超参数 (第 2 部分)

表 12 给出了 SIMPLER 与 ManiSkill 上使用的训练超参数。

表 12: SIMPLER 与 ManiSkill 上的超参数设置

参数	算法与任务									
	π_0									
	Eggplant	Carrot	Spoon	Cube	ManiSkill	Eggplant	Carrot	Spoon	Cube	ManiSkill
SFT train steps	1000	1000	1000	1000	1000	1000	1000	1000	1000	1000
RL train steps	40	40	40	130	150	40	40	40	70	150
Global batch size	2560	2560	2560	2560	5120	2560	2560	2560	2560	5120
Update epochs	4	4	4	4	4	4	4	4	4	5
Actor lr	5.6e-6	5.6e-6	5.6e-6	5.6e-6	7.91e-6	5.6e-6	5.6e-6	5.6e-6	5.6e-6	7.91e-6
Critic lr	1.1e-4	1.1e-4	1.1e-4	1.1e-4	1.55e-4	1.1e-4	1.1e-4	1.1e-4	1.1e-4	1.55e-4
Scheduler	False	False	False	False	False	False	False	False	False	False
Reward discount rate γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95	0.95
Clip ratio ϵ	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
Interaction steps	48	48	48	48	48	48	48	48	48	48
Parallel environments	256	256	256	256	320	256	256	256	256	320
Rollout epochs	1	1	1	1	1	1	1	1	1	1
Action prediction horizon H H'	8	8	8	8	8	8	8	8	8	8
Action replan horizon	5	5	5	5	5	5	5	5	5	5
Denoise steps	4	4	4	4	4	4	4	4	4	4
Noise level σ (Flow-SDE)	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
Max log-var (Flow-Noise)	0.16	0.16	0.16	0.16	0.16	0.10	0.10	0.10	0.10	0.10
Min log-var (Flow-Noise)	0.08	0.08	0.08	0.08	0.08	0.04	0.04	0.04	0.04	0.04
Entropy bonus (Flow-Noise)	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005	0.005

11.6 训练超参数（第 3 部分）

表 13 给出了 MetaWorld 与 CALVIN 基准上使用的超参数配置。

表 13: MetaWorld 与 CALVIN 基准上的超参数设置

参数	基准与模型			
	MetaWorld		CALVIN	
	π_0	$\pi_{0.5}$	π_0	$\pi_{0.5}$
训练轮数 (Train epochs)	450	450	100	100
全局 batch size	2048	2048	2048	2048
更新轮数 (Update epochs)	4	4	4	4
Actor 学习率 (lr)	1e-5	5e-6	5e-6	5e-6
Critic 学习率 (lr)	1e-4	1e-4	1e-4	1e-4
Scheduler	False	True	False	False
奖励折扣率 γ	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95
裁剪比例 ϵ	0.2	0.2	0.2	0.2
交互步数 (Interaction steps)	100	100	480	480
并行环境数 (Parallel environments)	64	64	64	64
Rollout 轮数 (Rollout epochs)	8	8	8	8
动作预测 horizon H	5	5	5	5
动作重规划 horizon H'	5	5	5	5
去噪步数 (Denoise steps)	5	5	5	5
噪声水平 σ (Flow-SDE)	0.5	0.5	0.5	0.5
最大 log-var (Flow-Noise)	0.10	0.10	0.16	0.16
最小 log-var (Flow-Noise)	0.04	0.04	0.08	0.08
熵奖励 (Entropy bonus, Flow-Noise)	0.005	0.005	0.005	0.005